Last updated: 2026-06-05 21:05:13 UTC | Student Version (No Solutions)

Linear Regression: Pipelines, Regularization, and Online Learning

Course: Machine Learning Labs
Lab: 01 — Linear Regression

Table of Contents

Estimated Time: 3–4 hours
Prerequisites: Python, NumPy, Pandas, introductory supervised learning

📦 Resolution

After the deadline, you will find the resolution at the following Notion link:

https://app.notion.com/p/R-Reg-lineal-python-intro-

2780573743578010af37de77f8e24bee?source=copy_link

Remember that there are several ways to solve the proposed activities in the labs. Use the provided resolution as a reference. If the resolution link is not public, request access at the same Notion link.

🚚 Deliverables

Don't forget to duplicate this notebook to be able to edit: File -> Save a copy in Drive

In this lab, you do not need to write a report in a separate document. Please complete all required activities within this Google Colab notebook. Remember that a notebook allows you to enter text elements similarly to a document processor. Once the proposed activities are completed, you must submit the following on the platform:

1. A PDF file generated in Google Colab from the menu "File" -> "Print".
2. The public Google Colab link. To do so, go to the share button and change the sharing settings to "Anyone with the link".

---

1. Setup and Imports

In [1]:
```
# Install / upgrade packages when running in Google Colab
try:
    import google.colab
    IN_COLAB = True
    import subprocess
    subprocess.run(['pip', 'install', '--quiet', '--upgrade',
                    'scikit-learn>=1.3', 'seaborn>=0.12'], check=Tru
    print('Colab environment ready.')
except ImportError:
    IN_COLAB = False
    print('Local environment detected.')
```

```
Local environment detected.
```

In [2]:
```
import os
import warnings
import urllib.request
import zipfile

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.datasets import fetch_california_housing
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import PolynomialFeatures, StandardScale
from sklearn.linear_model import Ridge, Lasso, SGDRegressor
from sklearn.model_selection import GridSearchCV, train_test_split,
from sklearn.metrics import mean_squared_error

import sklearn
print(f'scikit-learn : {sklearn.__version__}')
print(f'NumPy        : {np.__version__}')
print(f'Pandas       : {pd.__version__}')

warnings.filterwarnings('ignore')
```

```
scikit-learn : 1.7.2
NumPy        : 2.3.5
Pandas       : 2.3.3
```

In [3]:

```
# ── Reproducibility ──────────────────────────────────────────
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# ── Part 1: Pipeline / GridSearch ────────────────────────────
TEST_SIZE  = 0.2
CV_FOLDS   = 5
POLY_DEGREE = 3          # Exercise 1.1: try 1 or 3
ALPHAS     = np.logspace(-4, 4, 30)

# ── Part 2: Online Learning ──────────────────────────────────
CHUNK_SIZE     = 5_000
N_EPOCHS       = 10
MSD_TRAIN_ROWS = 463_715   # official UCI split: first 463,715 rows
MSD_TEST_ROWS  = 51_630    # last 51,630 rows are test (total = 515
MSD_DATA_FILE  = 'YearPredictionMSD.txt'
MSD_URL        = ('https://archive.ics.uci.edu/ml/machine-learning-
                  '/00203/YearPredictionMSD.txt.zip')

# ── Plotting defaults ────────────────────────────────────────
os.makedirs('assets', exist_ok=True)
plt.rcParams.update({
    'figure.figsize'   : (11, 5),
    'axes.spines.top'  : False,
    'axes.spines.right': False,
    'font.size'        : 11,
})
COLORS = ['steelblue', 'tomato', 'seagreen', 'darkorange', 'mediump

print('Configuration ready.')
```

```
Configuration ready.
```

---

2. Introduction

Linear regression is the foundation of supervised machine learning. Despite its simplicity, it remains one of the most widely deployed models in industry — from predicting house prices to forecasting energy demand. Mastering it means mastering the core ideas that every more complex model builds on: loss functions, gradient descent, regularization, and generalization.

This lab covers two perspectives on linear regression:

Part 1 — Sklearn Pipelines with Ridge Regression (instructor walkthrough)
You will build a complete machine-learning pipeline that chains polynomial feature expansion, feature scaling, and Ridge regression. You will use `GridSearchCV` to tune the regularization strength λ (alpha) and visualize the classic bias–variance tradeoff through a train/validation error curve.

Part 2 — Online Learning with SGDRegressor (student activity)
You will train a linear model on a dataset that is too large to hold in memory at once — the **Year Prediction MSD** dataset, a subset of the Million Song Dataset with 515 K songs and 90 audio features. You will use `SGDRegressor.partial_fit` to process the data in mini-batches, monitor convergence, and experimentally discover how the learning rate drives convergence or catastrophic divergence.

Learning Objectives

By the end of this lab you will be able to:

1. Explain why preprocessing steps must be wrapped in a `Pipeline` to prevent data leakage during cross-validation.
2. Build a multi-step sklearn `Pipeline` combining feature engineering, scaling, and a regularized estimator.
3. Use `GridSearchCV` to perform principled hyperparameter search and interpret the resulting validation curve.
4. Describe the difference between batch, stochastic, and mini-batch gradient descent.
5. Implement online / incremental learning with `SGDRegressor.partial_fit` on chunked data.
6. Diagnose learning-rate divergence and understand the role of learning-rate schedules.

---

3. Part 1 — Sklearn Pipelines with Ridge Regression

3.1 Dataset: California Housing

We use the **California Housing** dataset, derived from the 1990 US Census. Each row represents one census block group; the task is to predict the **median house value** (in units of $100,000).

Feature	Description
`MedInc`	Median household income in block
`HouseAge`	Median house age in block
`AveRooms`	Average number of rooms
`AveBedrms`	Average number of bedrooms
`Population`	Block population
`AveOccup`	Average occupancy per household
`Latitude`	Block latitude
`Longitude`	Block longitude

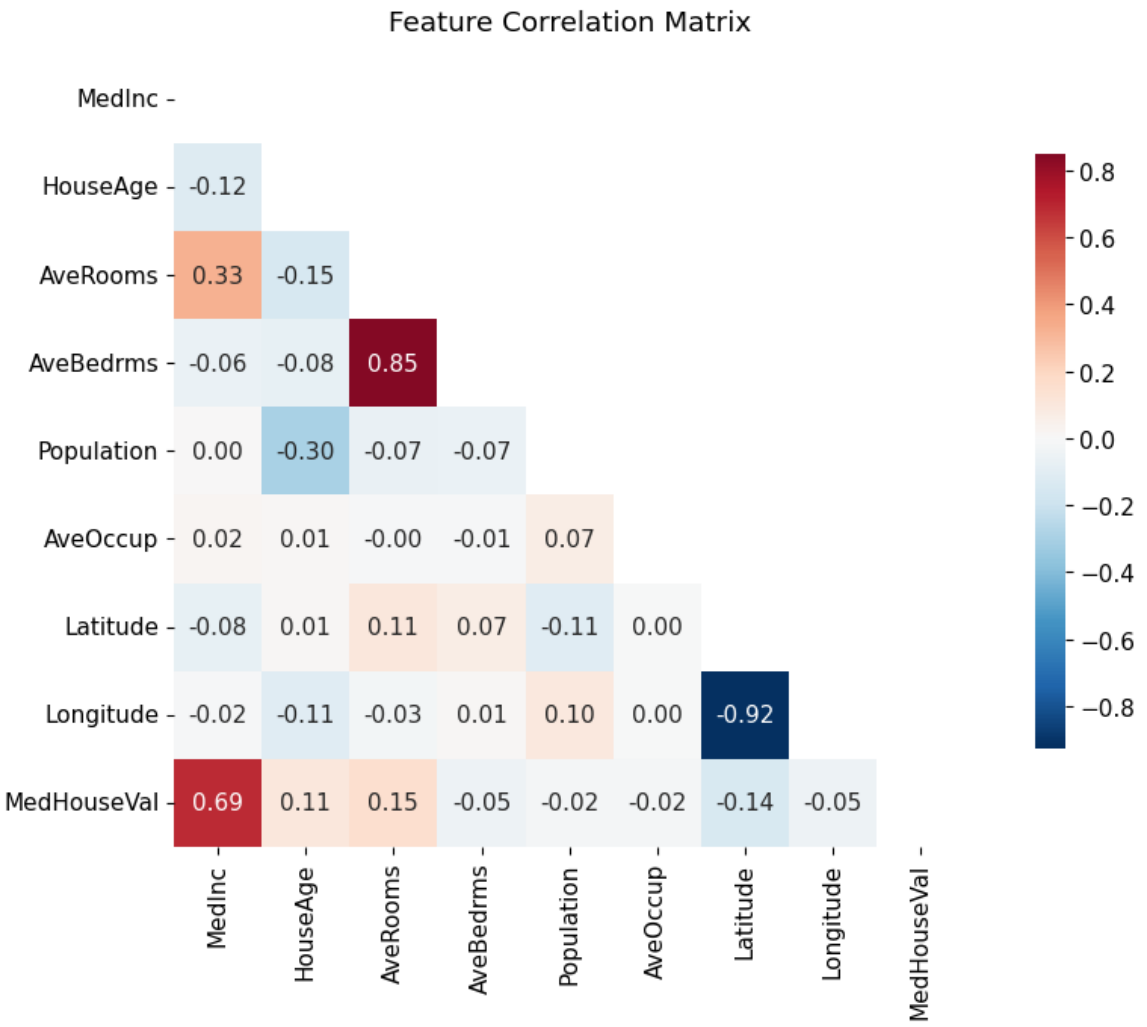
Target: `MedHouseVal` — median house value in $100 K (range 0.15 – 5.0).

In [4]:
```
housing = fetch_california_housing(as_frame=True)
df = housing.frame
X  = df.drop('MedHouseVal', axis=1)
y  = df['MedHouseVal']

print(f'Samples  : {df.shape[0]:,}')
print(f'Features : {X.shape[1]}')
print(f'Target   : MedHouseVal  min={y.min():.2f}  max={y.max():.2f}
print()
display(df.describe().round(2))
```

```
Samples  : 20,640
Features : 8
Target   : MedHouseVal  min=0.15  max=5.00  mean=2.07
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	I
count	20640.00	20640.00	20640.00	20640.00	20640.00	20640.00	2
mean	3.87	28.64	5.43	1.10	1425.48	3.07	
std	1.90	12.59	2.47	0.47	1132.46	10.39	
min	0.50	1.00	0.85	0.33	3.00	0.69	
25%	2.56	18.00	4.44	1.01	787.00	2.43	
50%	3.53	29.00	5.23	1.05	1166.00	2.82	
75%	4.74	37.00	6.05	1.10	1725.00	3.28	
max	15.00	52.00	141.91	34.07	35682.00	1243.33	

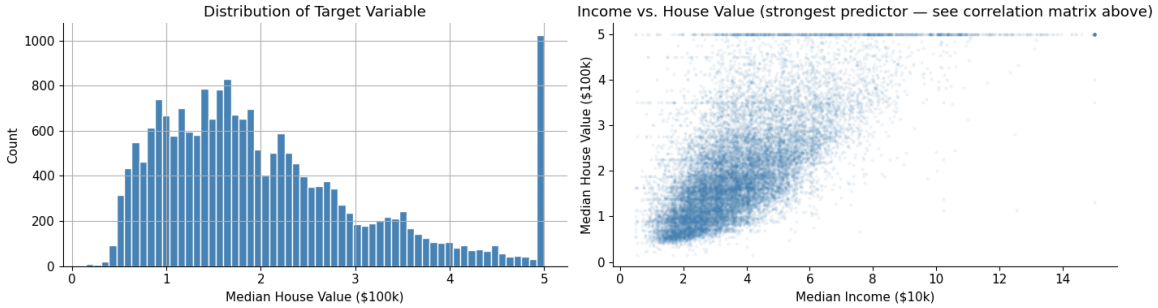
In [5]:
```
fig, ax = plt.subplots(figsize=(10, 7))
mask = np.triu(np.ones_like(df.corr(), dtype=bool))
sns.heatmap(
    df.corr(), mask=mask, annot=True, fmt='.2f',
    cmap='RdBu_r', center=0, square=True, ax=ax,
    cbar_kws={'shrink': 0.75}
)
ax.set_title('Feature Correlation Matrix', pad=12)
plt.tight_layout()
plt.savefig('assets/california_correlation.png', dpi=120, bbox_inch
plt.show()
```



Conclusion from the correlation matrix: `MedInc` (median income) has the highest correlation with the target (`MedHouseVal`) at **≈ 0.69**, well above the next strongest predictor `AveRooms` (≈ 0.15 after accounting for sign). The scatter plot below focuses on this dominant feature and reveals a non-linear curve — motivating polynomial feature expansion in Section 3.3.

In [6]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

y.hist(bins=60, ax=axes[0], color=COLORS[0], edgecolor='white')
axes[0].set_xlabel('Median House Value ($100k)')
axes[0].set_ylabel('Count')
axes[0].set_title('Distribution of Target Variable')
```

```
axes[1].scatter(df['MedInc'], y, alpha=0.06, s=4, color=COLORS[0])
axes[1].set_xlabel('Median Income ($10k)')
axes[1].set_ylabel('Median House Value ($100k)')
axes[1].set_title('Income vs. House Value (strongest predictor — se

plt.tight_layout()
plt.savefig('assets/california_eda.png', dpi=120, bbox_inches='tigh
plt.show()
```



The $500k Ceiling: Censored Data

The histogram above shows a visible spike at $500k (5.0 in $100k units). This is **not** a market phenomenon — it is a **data collection artifact**. In the 1990 census, median house values above $500k were all recorded as exactly $500k. Statistically, these are **right-censored observations**: we know the true value is ≥ $500k, but not by how much.

Training a regression model on censored targets creates systematic bias: the model is penalized for every block whose true value exceeds $500k, yet it has no signal to learn the correct magnitude. The result is a distorted loss landscape near the ceiling.

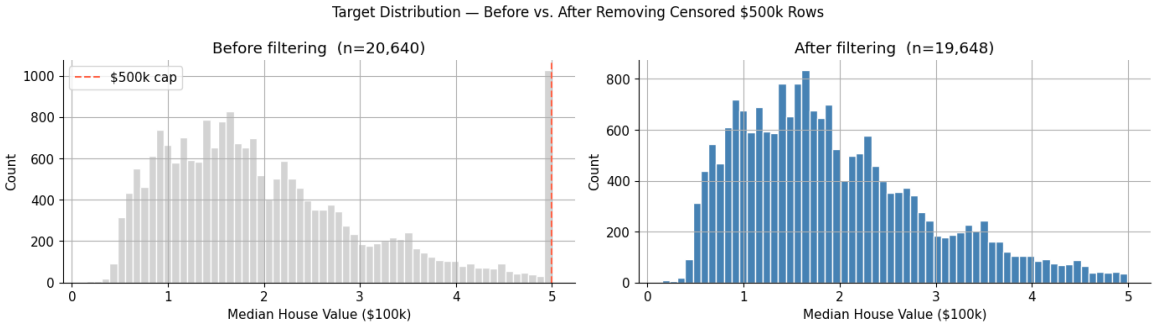
Decision: We filter out the ~965 censored rows (≈ 4.7% of data) and restrict the model to the $15k–$500k range. This is a **data-scope decision**, not data cleaning — the removed rows are not wrong, just incomplete.

In [7]:
```
n_total  = len(df)
n_capped = (df['MedHouseVal'] >= 5.0).sum()
print(f"Censored rows (MedHouseVal == $500k): {n_capped} / {n_total

df = df[df['MedHouseVal'] < 5.0].copy()
X  = df.drop('MedHouseVal', axis=1)
y  = df['MedHouseVal']

print(f"Dataset after filtering: {len(df):,} rows")
print(f"Target range now: ${y.min() * 100_000:,.0f} – ${y.max() * 1(
```

```
Censored rows (MedHouseVal == $500k): 992 / 20640 (4.8%)
Dataset after filtering: 19,648 rows
Target range now: $14,999 – $499,100
```

In [8]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
```

```
housing.frame['MedHouseVal'].hist(bins=60, ax=axes[0], color='light
axes[0].axvline(5.0, color=COLORS[1], linestyle='--', lw=1.5, label
axes[0].set_xlabel('Median House Value ($100k)')
axes[0].set_ylabel('Count')
axes[0].set_title('Before filtering  (n=20,640)')
axes[0].legend()

y.hist(bins=60, ax=axes[1], color=COLORS[0], edgecolor='white')
axes[1].set_xlabel('Median House Value ($100k)')
axes[1].set_ylabel('Count')
axes[1].set_title(f'After filtering  (n={len(df):,})')

plt.suptitle('Target Distribution — Before vs. After Removing Censo
plt.tight_layout()
plt.savefig('assets/target_distribution_filtered.png', dpi=120, bbo
plt.show()
```



3.1.1 Feature Skewness and Log Transforms

Raw features are rarely on a nice, symmetric scale — and that matters for linear models:

1. **Outliers dominate OLS.** Squared-error loss amplifies extreme values. `AveOccup` has a maximum of 1243 vs. a median of ~2.8 — a single such block can pull the fit away from the rest of the data.
2. **Log-linear relationships.** If house value grows *proportionally* with income (rather than additively), the true relationship is linear in *log(income)*. A model fit on raw income will struggle to capture this curve.

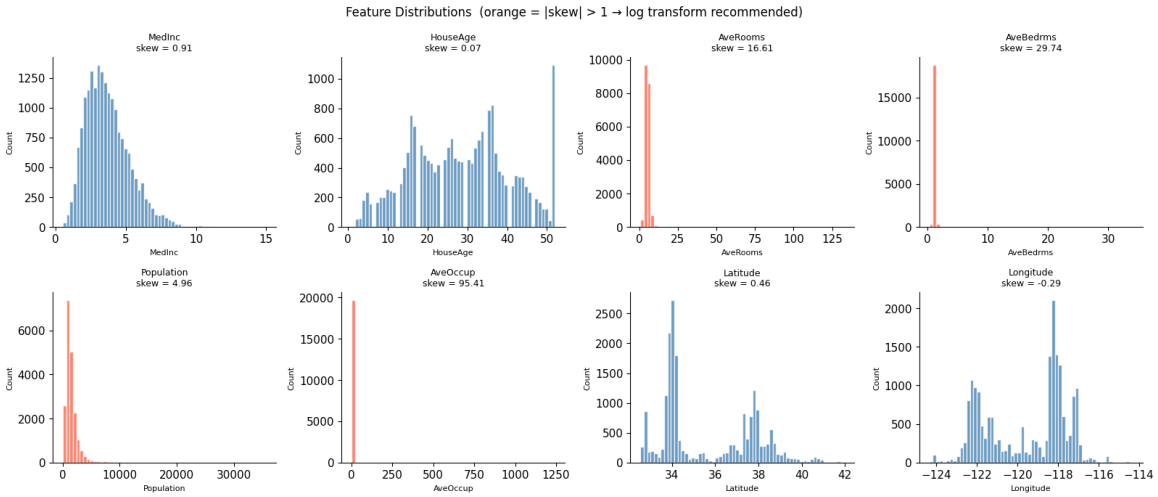
`log1p(x) = log(1 + x)` compresses right tails and safely handles zero values.

Rule of thumb: |skewness| > 1 → consider a log transform.

In [9]:
```
LOG_FEATURES  = ['MedInc', 'AveRooms', 'AveBedrms', 'Population', '
PASS_FEATURES = [c for c in X.columns if c not in LOG_FEATURES]

# Compute skewness for all features
feature_skewness = X.skew().sort_values(ascending=False)
print('Feature skewness (sorted):')
print(feature_skewness.round(2).to_string())
print()
```

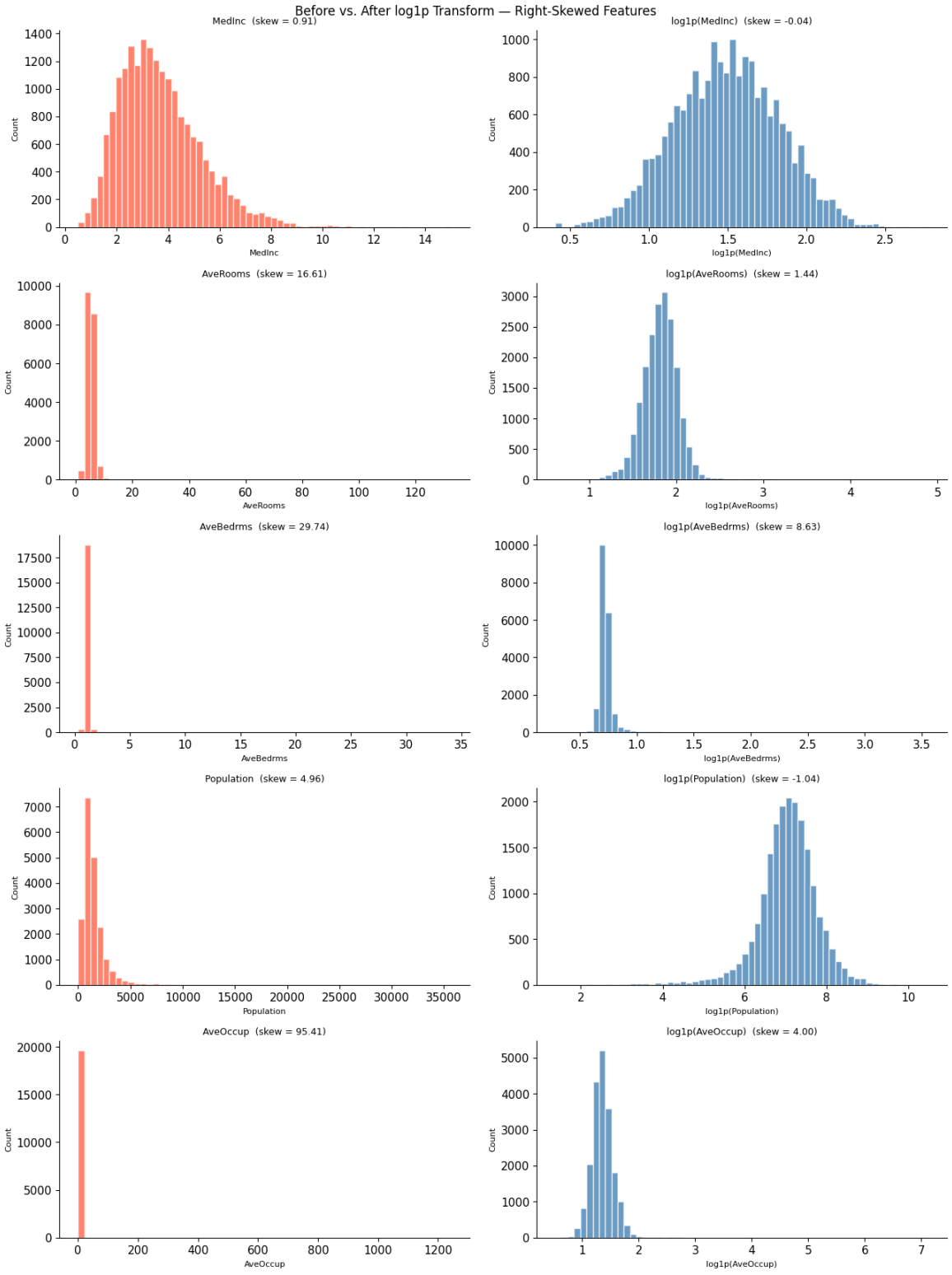
```
print('Features to log-transform (|skew| > 1):', LOG_FEATURES)
print('Features to passthrough               :', PASS_FEATURES)

fig, axes = plt.subplots(2, 4, figsize=(16, 7))
axes = axes.flatten()

for i, col in enumerate(X.columns):
    skew_val = X[col].skew()
    color = COLORS[1] if abs(skew_val) > 1 else COLORS[0]
    axes[i].hist(X[col], bins=60, color=color, edgecolor='white', a
    axes[i].set_title(f'{col}\nskew = {skew_val:.2f}', fontsize=9)
    axes[i].set_xlabel(col, fontsize=8)
    axes[i].set_ylabel('Count', fontsize=8)

plt.suptitle('Feature Distributions  (orange = |skew| > 1 → log tra
plt.tight_layout()
plt.savefig('assets/feature_distributions.png', dpi=120, bbox_inche
plt.show()
```

```
Feature skewness (sorted):
AveOccup      95.41
AveBedrms     29.74
AveRooms      16.61
Population     4.96
MedInc         0.91
Latitude       0.46
HouseAge       0.07
Longitude     -0.29

Features to log-transform (|skew| > 1): ['MedInc', 'AveRooms', 'AveB
edrms', 'Population', 'AveOccup']
Features to passthrough               : ['HouseAge', 'Latitude', 'L
ongitude']
```



In [10]:
```
fig, axes = plt.subplots(len(LOG_FEATURES), 2, figsize=(13, 3.5 * l

for i, col in enumerate(LOG_FEATURES):
    axes[i, 0].hist(X[col], bins=60, color=COLORS[1], edgecolor='wh
    axes[i, 0].set_title(f'{col}  (skew = {X[col].skew():.2f})', fo
    axes[i, 0].set_xlabel(col, fontsize=8)
    axes[i, 0].set_ylabel('Count', fontsize=8)

    log_col = np.log1p(X[col])
```

```
    axes[i, 1].hist(log_col, bins=60, color=COLORS[0], edgecolor='w
    axes[i, 1].set_title(f'log1p({col})  (skew = {log_col.skew():.2
    axes[i, 1].set_xlabel(f'log1p({col})', fontsize=8)
    axes[i, 1].set_ylabel('Count', fontsize=8)

plt.suptitle('Before vs. After log1p Transform — Right-Skewed Featu
plt.tight_layout()
plt.savefig('assets/log_transforms.png', dpi=120, bbox_inches='tigh
plt.show()
```



Remarks

- All five features become near-symmetric after log1p.

- The ColumnTransformer in Section 3.3 applies this automatically inside the pipeline.

3.2 Why Pipelines? Preventing Data Leakage

A common mistake is to fit the scaler (or any preprocessing step) on the **entire** dataset before splitting into train and test — or before cross-validation folds are formed. This is **data leakage**: the scaler learns the mean and standard deviation of the held-out samples, giving an overly optimistic evaluation.

```
# ❌ WRONG — scaler sees test data
scaler.fit(X_all)          # BUG: uses test
statistics
X_scaled = scaler.transform(X_all)
X_train, X_test = train_test_split(X_scaled, ...)

# ✅ RIGHT — Pipeline re-fits the scaler inside each
CV fold
pipe = Pipeline([('scaler', StandardScaler()),
('ridge', Ridge())])
cross_val_score(pipe, X_train, y_train, cv=5)
```

With a `Pipeline`, every call to `cross_val_score` or `GridSearchCV` will:

1. Split data into train/validation folds.
2. Call `fit_transform` on the training fold (scaler learns from train only).
3. Call `transform` on the validation fold (applies the train statistics).

This guarantees a valid estimate of generalization performance.

In [11]:
```
# Train / test split — done once, before any fitting
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE
)
print(f'Train: {X_train.shape[0]:,} samples')
print(f'Test : {X_test.shape[0]:,} samples')

# Quick baseline with a minimal pipeline (no poly features)
baseline_pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('ridge', Ridge()),
])
baseline_cv = -cross_val_score(
    baseline_pipe, X_train, y_train,
    cv=CV_FOLDS, scoring='neg_mean_squared_error', n_jobs=-1
)
baseline_rmse = np.sqrt(baseline_cv.mean())
print(f'\nBaseline CV RMSE (Ridge, no poly): {baseline_rmse:.4f} ($
```

```
Train: 15,718 samples
Test : 3,930 samples

Baseline CV RMSE (Ridge, no poly): 0.6353 ($100k)
```

3.3 Building the Ridge Pipeline

The income–value scatter (Section 3.1) shows a non-linear curve. We address this at two levels:

Step 1 — Log transforms (ColumnTransformer).
As shown in Section 3.1.1, five features (`MedInc` , `AveRooms` , `AveBedrms` , `Population` , `AveOccup`) are strongly right-skewed. We apply `log1p(x) = log(1 + x)` to each. `HouseAge` , `Latitude` , and `Longitude` pass through unchanged — they are either roughly symmetric or represent coordinates without a meaningful log scale.

Step 2 — Polynomial interaction features.
We use `interaction_only=True` , which adds only **pairwise products** (e.g. `log(MedInc)×Latitude` , capturing income–location pricing effects) and drops squared terms. This gives 8 original + C(8,2) = 28 interaction terms = **36 features**.

Step 3 — StandardScaler. Zero-mean, unit-variance scaling is required before Ridge, which penalises coefficient magnitude and is sensitive to feature scale.

Step 4 — Ridge regression. L2 penalty $\alpha \cdot \|w\|^2$ stabilises the fit against the many correlated interaction features.

Our full pipeline:

```
raw features  →  ColumnTransformer   →
PolynomialFeatures  →  StandardScaler  →  Ridge
(8 features)      log1p(5 features)
(interaction_only:    (zero mean,        (L2-
regularized
                  passthrough(3 feat.)    8 + 28 = 36
feat.)    unit variance)      linear model)
```

Why Ridge? Polynomial expansion creates many correlated features. OLS is then numerically unstable and overfits. Ridge's L2 penalty shrinks weights toward zero, reducing variance at the cost of a small bias.

In [12]:
```
log_preprocessor = ColumnTransformer([
    ('log',  FunctionTransformer(np.log1p, feature_names_out='one-t
    ('pass', 'passthrough', PASS_FEATURES),
], verbose_feature_names_out=False)
```

```
pipeline = Pipeline([
    ('log_transform', log_preprocessor),
    ('poly',   PolynomialFeatures(degree=POLY_DEGREE, include_bias=
    ('scaler', StandardScaler()),
    ('ridge',  Ridge()),
])

print('Log-transform features :', LOG_FEATURES)
print('Passthrough features   :', PASS_FEATURES)

n_poly_features = (
    PolynomialFeatures(degree=POLY_DEGREE, include_bias=False, inte
    .fit_transform(X_train)
    .shape[1]
)
print(f'\nDegree-{POLY_DEGREE} polynomial expansion: {X_train.shape
```

```
Log-transform features : ['MedInc', 'AveRooms', 'AveBedrms', 'Popula
tion', 'AveOccup']
Passthrough features   : ['HouseAge', 'Latitude', 'Longitude']

Degree-3 polynomial expansion: 8 → 164 features
```

3.4 Hyperparameter Tuning with GridSearchCV

The regularization strength **α** (called `ridge__alpha` in the pipeline) controls the bias–variance tradeoff:

- **Small α → low regularization**: the model fits training data tightly, potentially overfitting.
- **Large α → strong regularization**: weights are shrunk toward zero, potentially underfitting.
- **Sweet spot**: the α that minimises validation error.

`GridSearchCV` evaluates every candidate α using k-fold cross-validation and returns the one with the best mean validation score. The double underscore syntax `'ridge__alpha'` refers to the `alpha` parameter of the `ridge` step inside the pipeline.

In [13]:
```
param_grid = {'ridge__alpha': ALPHAS}

grid_search = GridSearchCV(
    pipeline,
    param_grid,
    cv=CV_FOLDS,
    scoring='neg_mean_squared_error',
    return_train_score=True,
    n_jobs=1,
    verbose=6,
)
```

```
print(f'Fitting {CV_FOLDS}-fold CV over {len(ALPHAS)} alpha values
grid_search.fit(X_train, y_train)
print('Done.')
print(f'\nBest alpha : {grid_search.best_params_["ridge__alpha"]:.4
print(f'Best CV RMSE: {np.sqrt(-grid_search.best_score_):.4f}  (imp
      f'{baseline_rmse - np.sqrt(-grid_search.best_score_):.4f})')
```

```
Fitting 5-fold CV over 30 alpha values ...
Fitting 5 folds for each of 30 candidates, totalling 150 fits
[CV 1/5] END ridge__alpha=0.0001;, score=(train=-0.264, test=-0.361)
total time=   0.0s
[CV 2/5] END ridge__alpha=0.0001;, score=(train=-0.258, test=-0.325)
total time=   0.0s
[CV 3/5] END ridge__alpha=0.0001;, score=(train=-0.264, test=-0.313)
total time=   0.0s
[CV 4/5] END ridge__alpha=0.0001;, score=(train=-0.270, test=-0.285)
total time=   0.0s
[CV 5/5] END ridge__alpha=0.0001;, score=(train=-0.264, test=-0.372)
total time=   0.0s
[CV 1/5] END ridge__alpha=0.00018873918221350977;, score=(train=-0.2
64, test=-0.345) total time=   0.0s
[CV 2/5] END ridge__alpha=0.00018873918221350977;, score=(train=-0.2
59, test=-0.324) total time=   0.0s
[CV 3/5] END ridge__alpha=0.00018873918221350977;, score=(train=-0.2
65, test=-0.312) total time=   0.0s
[CV 4/5] END ridge__alpha=0.00018873918221350977;, score=(train=-0.2
70, test=-0.281) total time=   0.0s
[CV 5/5] END ridge__alpha=0.00018873918221350977;, score=(train=-0.2
64, test=-0.356) total time=   0.0s
[CV 1/5] END ridge__alpha=0.0003562247890262444;, score=(train=-0.26
4, test=-0.332) total time=   0.0s
[CV 2/5] END ridge__alpha=0.0003562247890262444;, score=(train=-0.25
9, test=-0.322) total time=   0.0s
[CV 3/5] END ridge__alpha=0.0003562247890262444;, score=(train=-0.26
5, test=-0.310) total time=   0.0s
[CV 4/5] END ridge__alpha=0.0003562247890262444;, score=(train=-0.27
0, test=-0.276) total time=   0.0s
[CV 5/5] END ridge__alpha=0.0003562247890262444;, score=(train=-0.26
4, test=-0.339) total time=   0.0s
[CV 1/5] END ridge__alpha=0.0006723357536499335;, score=(train=-0.26
5, test=-0.322) total time=   0.0s
[CV 2/5] END ridge__alpha=0.0006723357536499335;, score=(train=-0.25
9, test=-0.320) total time=   0.0s
[CV 3/5] END ridge__alpha=0.0006723357536499335;, score=(train=-0.26
5, test=-0.306) total time=   0.0s
[CV 4/5] END ridge__alpha=0.0006723357536499335;, score=(train=-0.27
1, test=-0.271) total time=   0.0s
[CV 5/5] END ridge__alpha=0.0006723357536499335;, score=(train=-0.26
5, test=-0.323) total time=   0.0s
[CV 1/5] END ridge__alpha=0.0012689610031679222;, score=(train=-0.26
5, test=-0.315) total time=   0.1s
[CV 2/5] END ridge__alpha=0.0012689610031679222;, score=(train=-0.26
0, test=-0.317) total time=   0.0s
[CV 3/5] END ridge__alpha=0.0012689610031679222;, score=(train=-0.26
6, test=-0.301) total time=   0.0s
[CV 4/5] END ridge__alpha=0.0012689610031679222;, score=(train=-0.27
```

```
1, test=-0.267) total time=   0.0s
[CV 5/5] END ridge__alpha=0.0012689610031679222;, score=(train=-0.26
5, test=-0.310) total time=   0.0s
[CV 1/5] END ridge__alpha=0.002395026619987486;, score=(train=-0.26
6, test=-0.311) total time=   0.0s
[CV 2/5] END ridge__alpha=0.002395026619987486;, score=(train=-0.26
0, test=-0.314) total time=   0.0s
[CV 3/5] END ridge__alpha=0.002395026619987486;, score=(train=-0.26
6, test=-0.294) total time=   0.0s
[CV 4/5] END ridge__alpha=0.002395026619987486;, score=(train=-0.27
2, test=-0.265) total time=   0.0s
[CV 5/5] END ridge__alpha=0.002395026619987486;, score=(train=-0.26
6, test=-0.299) total time=   0.0s
[CV 1/5] END ridge__alpha=0.004520353656360241;, score=(train=-0.26
7, test=-0.307) total time=   0.1s
[CV 2/5] END ridge__alpha=0.004520353656360241;, score=(train=-0.26
1, test=-0.312) total time=   0.0s
[CV 3/5] END ridge__alpha=0.004520353656360241;, score=(train=-0.26
7, test=-0.288) total time=   0.0s
[CV 4/5] END ridge__alpha=0.004520353656360241;, score=(train=-0.27
3, test=-0.263) total time=   0.0s
[CV 5/5] END ridge__alpha=0.004520353656360241;, score=(train=-0.26
6, test=-0.292) total time=   0.0s
[CV 1/5] END ridge__alpha=0.008531678524172805;, score=(train=-0.26
7, test=-0.303) total time=   0.0s
[CV 2/5] END ridge__alpha=0.008531678524172805;, score=(train=-0.26
2, test=-0.310) total time=   0.0s
[CV 3/5] END ridge__alpha=0.008531678524172805;, score=(train=-0.26
8, test=-0.283) total time=   0.0s
[CV 4/5] END ridge__alpha=0.008531678524172805;, score=(train=-0.27
3, test=-0.262) total time=   0.0s
[CV 5/5] END ridge__alpha=0.008531678524172805;, score=(train=-0.26
7, test=-0.288) total time=   0.0s
[CV 1/5] END ridge__alpha=0.01610262027560939;, score=(train=-0.268,
test=-0.299) total time=   0.0s
[CV 2/5] END ridge__alpha=0.01610262027560939;, score=(train=-0.263,
test=-0.309) total time=   0.0s
[CV 3/5] END ridge__alpha=0.01610262027560939;, score=(train=-0.269,
test=-0.280) total time=   0.0s
[CV 4/5] END ridge__alpha=0.01610262027560939;, score=(train=-0.274,
test=-0.261) total time=   0.0s
[CV 5/5] END ridge__alpha=0.01610262027560939;, score=(train=-0.268,
test=-0.286) total time=   0.0s
[CV 1/5] END ridge__alpha=0.03039195382313198;, score=(train=-0.269,
test=-0.294) total time=   0.0s
[CV 2/5] END ridge__alpha=0.03039195382313198;, score=(train=-0.263,
test=-0.309) total time=   0.0s
[CV 3/5] END ridge__alpha=0.03039195382313198;, score=(train=-0.269,
test=-0.279) total time=   0.0s
[CV 4/5] END ridge__alpha=0.03039195382313198;, score=(train=-0.275,
test=-0.261) total time=   0.0s
[CV 5/5] END ridge__alpha=0.03039195382313198;, score=(train=-0.269,
test=-0.284) total time=   0.0s
[CV 1/5] END ridge__alpha=0.05736152510448681;, score=(train=-0.270,
test=-0.289) total time=   0.0s
[CV 2/5] END ridge__alpha=0.05736152510448681;, score=(train=-0.264,
```

```
test=-0.309) total time=   0.0s
[CV 3/5] END ridge__alpha=0.05736152510448681;, score=(train=-0.270,
test=-0.279) total time=   0.0s
[CV 4/5] END ridge__alpha=0.05736152510448681;, score=(train=-0.276,
test=-0.261) total time=   0.0s
[CV 5/5] END ridge__alpha=0.05736152510448681;, score=(train=-0.269,
test=-0.285) total time=   0.0s
[CV 1/5] END ridge__alpha=0.1082636733874054;, score=(train=-0.271,
test=-0.285) total time=   0.1s
[CV 2/5] END ridge__alpha=0.1082636733874054;, score=(train=-0.265,
test=-0.309) total time=   0.0s
[CV 3/5] END ridge__alpha=0.1082636733874054;, score=(train=-0.271,
test=-0.280) total time=   0.0s
[CV 4/5] END ridge__alpha=0.1082636733874054;, score=(train=-0.277,
test=-0.263) total time=   0.0s
[CV 5/5] END ridge__alpha=0.1082636733874054;, score=(train=-0.270,
test=-0.286) total time=   0.0s
[CV 1/5] END ridge__alpha=0.20433597178569418;, score=(train=-0.272,
test=-0.283) total time=   0.0s
[CV 2/5] END ridge__alpha=0.20433597178569418;, score=(train=-0.266,
test=-0.309) total time=   0.0s
[CV 3/5] END ridge__alpha=0.20433597178569418;, score=(train=-0.273,
test=-0.282) total time=   0.0s
[CV 4/5] END ridge__alpha=0.20433597178569418;, score=(train=-0.278,
test=-0.264) total time=   0.1s
[CV 5/5] END ridge__alpha=0.20433597178569418;, score=(train=-0.271,
test=-0.288) total time=   0.0s
[CV 1/5] END ridge__alpha=0.38566204211634725;, score=(train=-0.274,
test=-0.283) total time=   0.0s
[CV 2/5] END ridge__alpha=0.38566204211634725;, score=(train=-0.268,
test=-0.308) total time=   0.0s
[CV 3/5] END ridge__alpha=0.38566204211634725;, score=(train=-0.274,
test=-0.284) total time=   0.0s
[CV 4/5] END ridge__alpha=0.38566204211634725;, score=(train=-0.280,
test=-0.264) total time=   0.0s
[CV 5/5] END ridge__alpha=0.38566204211634725;, score=(train=-0.273,
test=-0.288) total time=   0.0s
[CV 1/5] END ridge__alpha=0.7278953843983146;, score=(train=-0.276,
test=-0.285) total time=   0.0s
[CV 2/5] END ridge__alpha=0.7278953843983146;, score=(train=-0.271,
test=-0.308) total time=   0.0s
[CV 3/5] END ridge__alpha=0.7278953843983146;, score=(train=-0.277,
test=-0.286) total time=   0.0s
[CV 4/5] END ridge__alpha=0.7278953843983146;, score=(train=-0.282,
test=-0.265) total time=   0.0s
[CV 5/5] END ridge__alpha=0.7278953843983146;, score=(train=-0.275,
test=-0.287) total time=   0.0s
[CV 1/5] END ridge__alpha=1.3738237958832638;, score=(train=-0.279,
test=-0.288) total time=   0.0s
[CV 2/5] END ridge__alpha=1.3738237958832638;, score=(train=-0.274,
test=-0.309) total time=   0.0s
[CV 3/5] END ridge__alpha=1.3738237958832638;, score=(train=-0.280,
test=-0.289) total time=   0.0s
[CV 4/5] END ridge__alpha=1.3738237958832638;, score=(train=-0.285,
test=-0.266) total time=   0.0s
[CV 5/5] END ridge__alpha=1.3738237958832638;, score=(train=-0.278,
```

```
test=-0.288) total time=   0.0s
[CV 1/5] END ridge__alpha=2.592943797404667;, score=(train=-0.282, t
est=-0.291) total time=   0.0s
[CV 2/5] END ridge__alpha=2.592943797404667;, score=(train=-0.277, t
est=-0.311) total time=   0.0s
[CV 3/5] END ridge__alpha=2.592943797404667;, score=(train=-0.283, t
est=-0.292) total time=   0.0s
[CV 4/5] END ridge__alpha=2.592943797404667;, score=(train=-0.288, t
est=-0.268) total time=   0.0s
[CV 5/5] END ridge__alpha=2.592943797404667;, score=(train=-0.282, t
est=-0.290) total time=   0.0s
[CV 1/5] END ridge__alpha=4.893900918477489;, score=(train=-0.286, t
est=-0.295) total time=   0.0s
[CV 2/5] END ridge__alpha=4.893900918477489;, score=(train=-0.281, t
est=-0.315) total time=   0.0s
[CV 3/5] END ridge__alpha=4.893900918477489;, score=(train=-0.287, t
est=-0.293) total time=   0.0s
[CV 4/5] END ridge__alpha=4.893900918477489;, score=(train=-0.291, t
est=-0.272) total time=   0.0s
[CV 5/5] END ridge__alpha=4.893900918477489;, score=(train=-0.286, t
est=-0.293) total time=   0.0s
[CV 1/5] END ridge__alpha=9.236708571873866;, score=(train=-0.289, t
est=-0.298) total time=   0.0s
[CV 2/5] END ridge__alpha=9.236708571873866;, score=(train=-0.285, t
est=-0.319) total time=   0.0s
[CV 3/5] END ridge__alpha=9.236708571873866;, score=(train=-0.291, t
est=-0.294) total time=   0.0s
[CV 4/5] END ridge__alpha=9.236708571873866;, score=(train=-0.295, t
est=-0.277) total time=   0.0s
[CV 5/5] END ridge__alpha=9.236708571873866;, score=(train=-0.289, t
est=-0.295) total time=   0.0s
[CV 1/5] END ridge__alpha=17.433288221999874;, score=(train=-0.293,
test=-0.301) total time=   0.0s
[CV 2/5] END ridge__alpha=17.433288221999874;, score=(train=-0.288,
test=-0.323) total time=   0.0s
[CV 3/5] END ridge__alpha=17.433288221999874;, score=(train=-0.294,
test=-0.295) total time=   0.0s
[CV 4/5] END ridge__alpha=17.433288221999874;, score=(train=-0.298,
test=-0.281) total time=   0.0s
[CV 5/5] END ridge__alpha=17.433288221999874;, score=(train=-0.293,
test=-0.297) total time=   0.0s
[CV 1/5] END ridge__alpha=32.90344562312671;, score=(train=-0.296, t
est=-0.304) total time=   0.0s
[CV 2/5] END ridge__alpha=32.90344562312671;, score=(train=-0.291, t
est=-0.326) total time=   0.0s
[CV 3/5] END ridge__alpha=32.90344562312671;, score=(train=-0.298, t
est=-0.296) total time=   0.0s
[CV 4/5] END ridge__alpha=32.90344562312671;, score=(train=-0.302, t
est=-0.285) total time=   0.0s
[CV 5/5] END ridge__alpha=32.90344562312671;, score=(train=-0.296, t
est=-0.300) total time=   0.0s
[CV 1/5] END ridge__alpha=62.10169418915616;, score=(train=-0.299, t
est=-0.306) total time=   0.0s
[CV 2/5] END ridge__alpha=62.10169418915616;, score=(train=-0.294, t
est=-0.330) total time=   0.0s
[CV 3/5] END ridge__alpha=62.10169418915616;, score=(train=-0.301, t
```

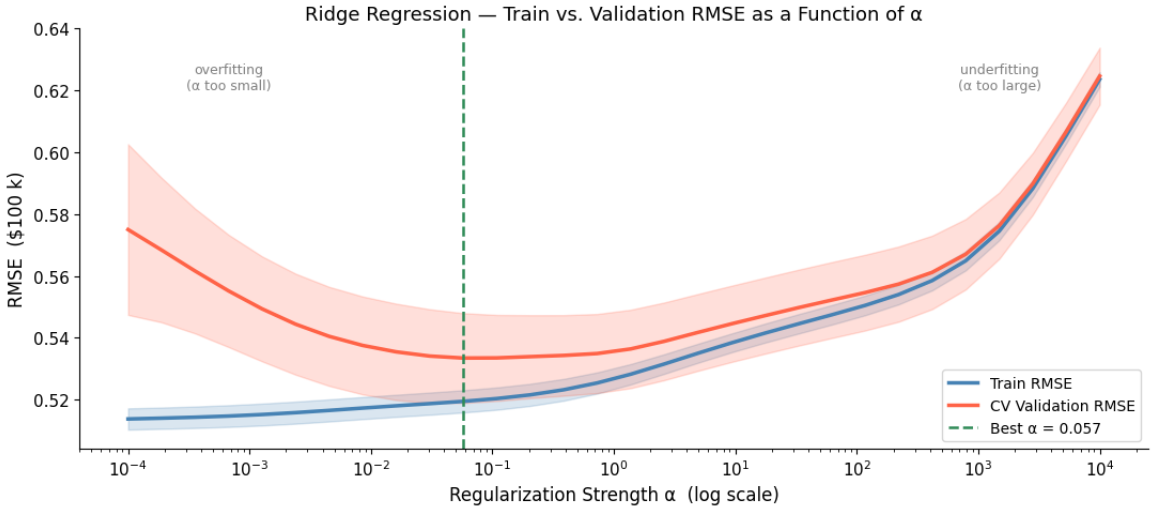
```
est=-0.298) total time=   0.0s
[CV 4/5] END ridge__alpha=62.10169418915616;, score=(train=-0.305, t
est=-0.288) total time=   0.0s
[CV 5/5] END ridge__alpha=62.10169418915616;, score=(train=-0.299, t
est=-0.303) total time=   0.0s
[CV 1/5] END ridge__alpha=117.21022975334793;, score=(train=-0.303,
test=-0.309) total time=   0.0s
[CV 2/5] END ridge__alpha=117.21022975334793;, score=(train=-0.297,
test=-0.332) total time=   0.0s
[CV 3/5] END ridge__alpha=117.21022975334793;, score=(train=-0.304,
test=-0.301) total time=   0.0s
[CV 4/5] END ridge__alpha=117.21022975334793;, score=(train=-0.308,
test=-0.291) total time=   0.0s
[CV 5/5] END ridge__alpha=117.21022975334793;, score=(train=-0.303,
test=-0.306) total time=   0.0s
[CV 1/5] END ridge__alpha=221.22162910704503;, score=(train=-0.307,
test=-0.312) total time=   0.0s
[CV 2/5] END ridge__alpha=221.22162910704503;, score=(train=-0.301,
test=-0.335) total time=   0.0s
[CV 3/5] END ridge__alpha=221.22162910704503;, score=(train=-0.308,
test=-0.305) total time=   0.0s
[CV 4/5] END ridge__alpha=221.22162910704503;, score=(train=-0.312,
test=-0.293) total time=   0.0s
[CV 5/5] END ridge__alpha=221.22162910704503;, score=(train=-0.307,
test=-0.309) total time=   0.0s
[CV 1/5] END ridge__alpha=417.53189365604004;, score=(train=-0.312,
test=-0.316) total time=   0.0s
[CV 2/5] END ridge__alpha=417.53189365604004;, score=(train=-0.306,
test=-0.338) total time=   0.0s
[CV 3/5] END ridge__alpha=417.53189365604004;, score=(train=-0.313,
test=-0.309) total time=   0.0s
[CV 4/5] END ridge__alpha=417.53189365604004;, score=(train=-0.317,
test=-0.297) total time=   0.0s
[CV 5/5] END ridge__alpha=417.53189365604004;, score=(train=-0.312,
test=-0.315) total time=   0.0s
[CV 1/5] END ridge__alpha=788.0462815669904;, score=(train=-0.319, t
est=-0.323) total time=   0.0s
[CV 2/5] END ridge__alpha=788.0462815669904;, score=(train=-0.313, t
est=-0.344) total time=   0.0s
[CV 3/5] END ridge__alpha=788.0462815669904;, score=(train=-0.321, t
est=-0.315) total time=   0.0s
[CV 4/5] END ridge__alpha=788.0462815669904;, score=(train=-0.324, t
est=-0.304) total time=   0.0s
[CV 5/5] END ridge__alpha=788.0462815669904;, score=(train=-0.319, t
est=-0.323) total time=   0.0s
[CV 1/5] END ridge__alpha=1487.3521072935118;, score=(train=-0.330,
test=-0.333) total time=   0.0s
[CV 2/5] END ridge__alpha=1487.3521072935118;, score=(train=-0.324,
test=-0.353) total time=   0.0s
[CV 3/5] END ridge__alpha=1487.3521072935118;, score=(train=-0.332,
test=-0.326) total time=   0.0s
[CV 4/5] END ridge__alpha=1487.3521072935118;, score=(train=-0.335,
test=-0.315) total time=   0.0s
[CV 5/5] END ridge__alpha=1487.3521072935118;, score=(train=-0.330,
test=-0.335) total time=   0.0s
[CV 1/5] END ridge__alpha=2807.2162039411755;, score=(train=-0.346,
```

```
test=-0.347) total time=   0.0s
[CV 2/5] END ridge__alpha=2807.2162039411755;, score=(train=-0.341,
test=-0.367) total time=   0.0s
[CV 3/5] END ridge__alpha=2807.2162039411755;, score=(train=-0.348,
test=-0.341) total time=   0.0s
[CV 4/5] END ridge__alpha=2807.2162039411755;, score=(train=-0.351,
test=-0.331) total time=   0.0s
[CV 5/5] END ridge__alpha=2807.2162039411755;, score=(train=-0.345,
test=-0.353) total time=   0.0s
[CV 1/5] END ridge__alpha=5298.316906283702;, score=(train=-0.366, t
est=-0.366) total time=   0.0s
[CV 2/5] END ridge__alpha=5298.316906283702;, score=(train=-0.362, t
est=-0.386) total time=   0.0s
[CV 3/5] END ridge__alpha=5298.316906283702;, score=(train=-0.368, t
est=-0.361) total time=   0.0s
[CV 4/5] END ridge__alpha=5298.316906283702;, score=(train=-0.371, t
est=-0.352) total time=   0.0s
[CV 5/5] END ridge__alpha=5298.316906283702;, score=(train=-0.365, t
est=-0.375) total time=   0.0s
[CV 1/5] END ridge__alpha=10000.0;, score=(train=-0.389, test=-0.38
7) total time=   0.0s
[CV 2/5] END ridge__alpha=10000.0;, score=(train=-0.385, test=-0.40
7) total time=   0.0s
[CV 3/5] END ridge__alpha=10000.0;, score=(train=-0.391, test=-0.38
3) total time=   0.0s
[CV 4/5] END ridge__alpha=10000.0;, score=(train=-0.393, test=-0.37
5) total time=   0.0s
[CV 5/5] END ridge__alpha=10000.0;, score=(train=-0.387, test=-0.39
9) total time=   0.0s
Done.

Best alpha : 0.0574
Best CV RMSE: 0.5336  (improvement over baseline: 0.1017)
```

In [14]:

```python
# ── Extract per-fold RMSE for accurate error bands ──────────────
results       = pd.DataFrame(grid_search.cv_results_)
alphas_tested = results['param_ridge__alpha'].astype(float).values

fold_train_rmse = np.array([
    np.sqrt(-results[f'split{k}_train_score'].values)
    for k in range(CV_FOLDS)
])  # shape (CV_FOLDS, n_alphas)

fold_val_rmse = np.array([
    np.sqrt(-results[f'split{k}_test_score'].values)
    for k in range(CV_FOLDS)
])

train_rmse     = fold_train_rmse.mean(axis=0)
train_rmse_std = fold_train_rmse.std(axis=0)
cv_rmse        = fold_val_rmse.mean(axis=0)
cv_rmse_std    = fold_val_rmse.std(axis=0)

best_alpha = grid_search.best_params_['ridge__alpha']

# ── Plot ────────────────────────────────────────────────────────
```

```
fig, ax = plt.subplots(figsize=(11, 5))

ax.semilogx(alphas_tested, train_rmse, color=COLORS[0], lw=2.5, lab
ax.fill_between(alphas_tested,
                train_rmse - train_rmse_std, train_rmse + train_rms
                alpha=0.2, color=COLORS[0])

ax.semilogx(alphas_tested, cv_rmse, color=COLORS[1], lw=2.5, label=
ax.fill_between(alphas_tested,
                cv_rmse - cv_rmse_std, cv_rmse + cv_rmse_std,
                alpha=0.2, color=COLORS[1])

ax.axvline(best_alpha, color=COLORS[2], linestyle='--', lw=1.8,
           label=f'Best α = {best_alpha:.3f}')

# Annotate regions
ylo, yhi = ax.get_ylim()
ytext = ylo + (yhi - ylo) * 0.92
ax.text(alphas_tested[3],  ytext, 'overfitting\n(α too small)',
        ha='center', va='top', fontsize=9, color='gray')
ax.text(alphas_tested[-4], ytext, 'underfitting\n(α too large)',
        ha='center', va='top', fontsize=9, color='gray')

ax.set_xlabel('Regularization Strength α  (log scale)', fontsize=12
ax.set_ylabel('RMSE  ($100 k)', fontsize=12)
ax.set_title('Ridge Regression — Train vs. Validation RMSE as a Fun
ax.legend(fontsize=10)
plt.tight_layout()
plt.savefig('assets/ridge_validation_curve.png', dpi=130, bbox_inch
plt.show()
```



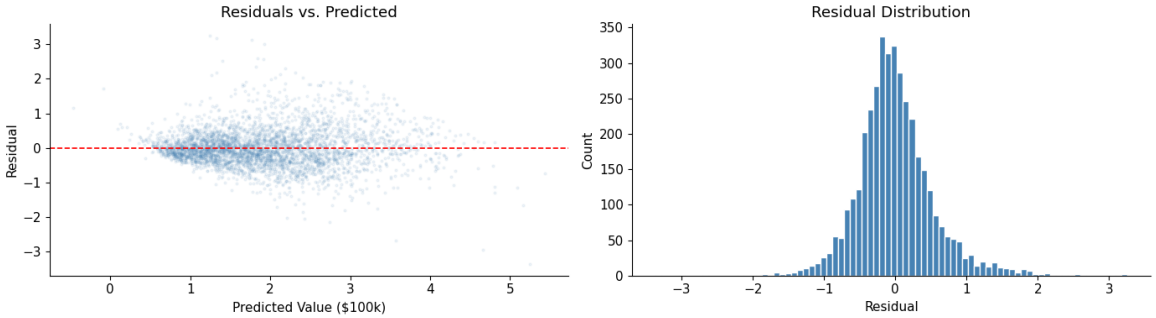
Interpretation:

- Left (small α): train RMSE low, CV RMSE high → overfitting
- Right (large α): both RMSEs high → underfitting
- Minimum of CV RMSE curve = best regularization strength

3.5 Model Evaluation on the Test Set

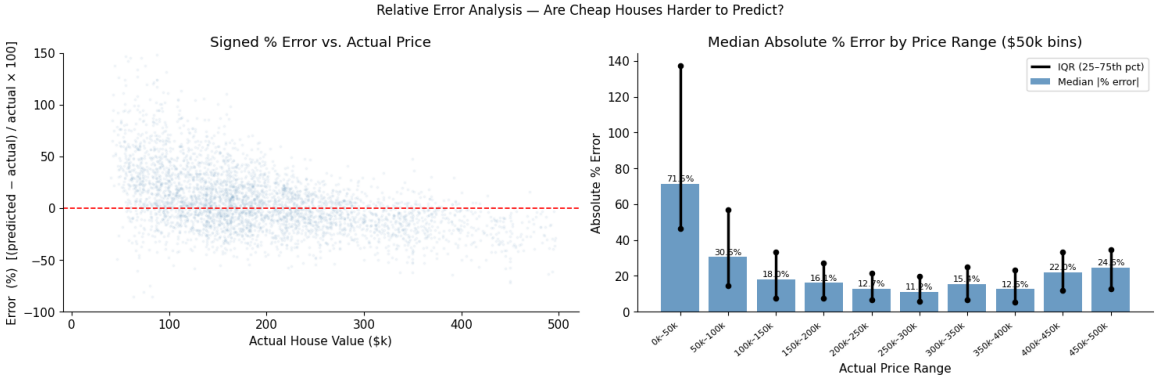
In [15]:
```
best_model = grid_search.best_estimator_
y_pred     = best_model.predict(X_test)

test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print(f'Test RMSE (Ridge, degree-{POLY_DEGREE}): {test_rmse:.4f}  ($
print(f'In dollar terms: ± ${test_rmse * 100_000:,.0f}')

# Residual plot
residuals = y_test - y_pred
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

axes[0].scatter(y_pred, residuals, alpha=0.08, s=4, color=COLORS[0]
axes[0].axhline(0, color='red', lw=1.2, linestyle='--')
axes[0].set_xlabel('Predicted Value ($100k)')
axes[0].set_ylabel('Residual')
axes[0].set_title('Residuals vs. Predicted')

axes[1].hist(residuals, bins=80, color=COLORS[0], edgecolor='white'
axes[1].set_xlabel('Residual')
axes[1].set_ylabel('Count')
axes[1].set_title('Residual Distribution')

plt.tight_layout()
plt.savefig('assets/ridge_residuals.png', dpi=120, bbox_inches='tig
plt.show()
```

```
Test RMSE (Ridge, degree-3): 0.5391  ($100k)
In dollar terms: ± $53,906
```



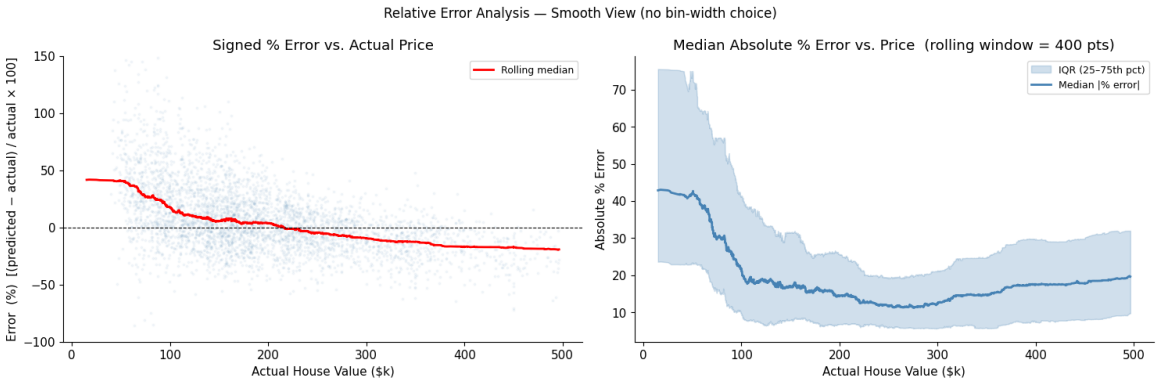
In [16]:
```
bin_width_k = 50  # bin width in $k — try 25, 50, 100 to see coarse

pct_error = (y_pred - y_test) / y_test * 100          # signed: + =
abs_pct_error = np.abs(pct_error)

price_dollars = y_test * 100_000
bin_width = bin_width_k * 1_000
max_price = int(np.ceil(price_dollars.max() / bin_width) * bin_widt
bins = np.arange(0, max_price, bin_width)
labels = [f'${int(b/1000)}k–${int((b+bin_width)/1000)}k' for b in b
price_bin = pd.cut(price_dollars, bins=bins, labels=labels)

bin_stats = pd.DataFrame({
    'abs_pct_error': abs_pct_error,
    'pct_error': pct_error,
    'price_bin': price_bin,
}).groupby('price_bin', observed=True).agg(
    median_abs_pct=('abs_pct_error', 'median'),
```

```
    p25=('abs_pct_error', lambda x: np.percentile(x, 25)),
    p75=('abs_pct_error', lambda x: np.percentile(x, 75)),
    n=('abs_pct_error', 'count'),
)

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Left: scatter of signed % error vs actual price
axes[0].scatter(price_dollars / 1000, pct_error, alpha=0.05, s=3, c
axes[0].axhline(0, color='red', lw=1.2, linestyle='--')
axes[0].set_xlabel('Actual House Value ($k)')
axes[0].set_ylabel('Error  (%)  [(predicted - actual) / actual × 10
axes[0].set_title('Signed % Error vs. Actual Price')
axes[0].set_ylim(-100, 150)

# Right: binned median absolute % error with IQR band
x_pos = np.arange(len(bin_stats))
axes[1].bar(x_pos, bin_stats['median_abs_pct'], color=COLORS[0], al
axes[1].vlines(x_pos,
               bin_stats['p25'], bin_stats['p75'],
               color='black', lw=2.5, label='IQR (25–75th pct)')
axes[1].scatter(x_pos, bin_stats['p75'], color='black', s=20, zorde
axes[1].scatter(x_pos, bin_stats['p25'], color='black', s=20, zorde

for i, (idx, row) in enumerate(bin_stats.iterrows()):
    axes[1].text(i, row['median_abs_pct'] + 0.5, f"{row['median_abs
                 ha='center', va='bottom', fontsize=8)

axes[1].set_xticks(x_pos)
axes[1].set_xticklabels(bin_stats.index, rotation=40, ha='right', f
axes[1].set_xlabel('Actual Price Range')
axes[1].set_ylabel('Absolute % Error')
axes[1].set_title(f'Median Absolute % Error by Price Range (${bin_w
axes[1].legend(fontsize=9)

plt.suptitle('Relative Error Analysis — Are Cheap Houses Harder to
plt.tight_layout()
plt.savefig('assets/relative_error_by_price.png', dpi=130, bbox_incl
plt.show()

mape = abs_pct_error.mean()
print(f'MAPE (mean absolute % error): {mape:.1f}%')
print(f'Median absolute % error     : {abs_pct_error.median():.1f}%
```




```
MAPE (mean absolute % error): 24.5%
Median absolute % error     : 17.1%
```

In [17]:
```
SMOOTH_WINDOW = 400  # number of nearest neighbors (by price) used

# Sort test points by actual price so the rolling window is price-o
sort_idx = np.argsort(price_dollars.values)
price_s  = price_dollars.values[sort_idx] / 1_000   # in $k
pct_s    = pct_error.values[sort_idx]
abs_s    = abs_pct_error.values[sort_idx]

kw = dict(window=SMOOTH_WINDOW, center=True, min_periods=50)
med_signed = pd.Series(pct_s).rolling(**kw).median()
med_abs    = pd.Series(abs_s).rolling(**kw).median()
p25        = pd.Series(abs_s).rolling(**kw).quantile(0.25)
p75        = pd.Series(abs_s).rolling(**kw).quantile(0.75)

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Left: scatter + rolling median of signed % error
axes[0].scatter(price_s, pct_s, alpha=0.05, s=3, color=COLORS[0])
axes[0].plot(price_s, med_signed, color='red', lw=2, label='Rolling
axes[0].axhline(0, color='black', lw=0.8, linestyle='--')
axes[0].set_ylim(-100, 150)
axes[0].set_xlabel('Actual House Value ($k)')
axes[0].set_ylabel('Error  (%)  [(predicted - actual) / actual × 10
axes[0].set_title('Signed % Error vs. Actual Price')
axes[0].legend(fontsize=9)

# Right: rolling median absolute % error + IQR band
axes[1].fill_between(price_s, p25, p75, alpha=0.25, color=COLORS[0]
axes[1].plot(price_s, med_abs, color=COLORS[0], lw=2, label='Median
axes[1].set_xlabel('Actual House Value ($k)')
axes[1].set_ylabel('Absolute % Error')
axes[1].set_title(f'Median Absolute % Error vs. Price  (rolling win
axes[1].legend(fontsize=9)

plt.suptitle('Relative Error Analysis — Smooth View (no bin-width c
plt.tight_layout()
plt.savefig('assets/relative_error_smooth.png', dpi=130, bbox_inche
plt.show()
```



The smooth curve above shows the same signal as the bar chart, but without a bin-width choice. Both views are complementary: the bars give precise per-range numbers; the smooth curve makes the overall trend and local anomalies (the $300k bump, the rise above $400k) easier to read at a glance.

What does this plot tell us?

The right panel reveals a clear pattern: a **steep monotonic decline** from the cheapest houses down to a sweet spot around $250k–$300k, followed by a gradual rise at the expensive end.

Very cheap houses ($0–$50k) — 71.5% median error, IQR reaching ~140%

This is an extreme segment. Almost no California census blocks reported a median value below $50k in 1990, so the model has barely any training signal here. The enormous IQR (the whisker reaches ~140%) tells us predictions are nearly random — the few data points are so heterogeneous that even their central tendency is uncertain.

The monotonic decline ($50k → $250k) — 30.5% → 11.8%

Error falls steadily as house value increases. Two effects drive this:

1. *Data density.* The training distribution is centered around $150k–$300k. As we move right across this range, the model sees progressively more examples and fits better.
2. *Percentage amplification.* A fixed absolute dollar error of, say, $15k is 30% of a $50k house but only 6% of a $250k house. Even if prediction quality were identical across the range, the relative error metric would still show this downward slope.

Sweet spot ($250k–$300k) — 11.8% median error

This is where the model performs best: the highest concentration of training data, the most consistent income-location-price relationships, and the lowest percentage amplification effect. Absolute dollar errors in this bin are likely similar to neighboring bins — the relative metric just favors this range.

The slight bump at $300k–$350k (15.4%) then recovery at $350k–$400k (12.5%)

This small anomaly is worth noting. One plausible explanation is a feature interaction: blocks in the $300k–$350k range may straddle a geographic boundary (e.g., the edge of a high-demand coastal county vs. an inland suburb) where the model's polynomial features produce a local misfit. With only 8 census variables, the model cannot distinguish otherwise-similar blocks that sit on opposite sides of a neighborhood boundary.

Expensive houses ($400k–$500k) — rising to 22–24.5%

Error climbs again at the high end for structural reasons:

1. *Proximity to the censoring boundary.* We removed the hard-capped $500k rows, but blocks just below that cap may still carry artifacts of the censoring process — their features can be systematically unusual relative to blocks priced well below the ceiling.
2. *Luxury non-linearities.* High-end prices reflect factors the 8 census features cannot capture: school district prestige, architectural quality, micro-neighborhood cachet. The model undershoots these values.
3. *Sparse data.* Like the cheap end, few training examples exist here, so the model cannot specialize.

Left panel — signed errors

The scatter confirms the directional story: cheap houses are overwhelmingly **over-predicted** (positive signed error, large upward spread), while expensive houses cluster just below the zero line, reflecting systematic **under-prediction**. This is the classic **regression-to-the-mean** bias — a linear model trained on an asymmetric distribution anchors predictions toward the bulk of the data, dragging cheap predictions up and expensive predictions down.

Takeaway: relative (percentage) error is a more honest evaluation metric than RMSE for datasets with wide price ranges. RMSE would mask the poor performance at the extremes because those segments are numerically small and infrequent.

Exercise 1.1 — Effect of Polynomial Degree (Basic)

So far we used `POLY_DEGREE = 2`. In this exercise you will compare degree 1, 2, and 3 to see how polynomial degree interacts with regularization.

Tasks:

1. For each degree in `[1, 2, 3]`, build a Ridge pipeline, run `GridSearchCV` with the same `ALPHAS`, and record the best CV RMSE and the corresponding best α.
2. Print a summary table.
3. Answer: does a higher polynomial degree always improve test performance? Why or why not?

In [30]:
```
from sklearn.metrics import mean_squared_error
import numpy as np
import pandas as pd

results = []

for degree in [1, 2, 3]:
```

```
    # update pipeline degree
    model = Pipeline([
        ('log_transform', log_preprocessor),
        ('poly', PolynomialFeatures(degree=degree, include_bias=Fal
        ('scaler', StandardScaler()),
        ('ridge', Ridge())
    ])

    grid = GridSearchCV(
        model,
        param_grid={'ridge__alpha': ALPHAS},
        cv=CV_FOLDS,
        scoring='neg_root_mean_squared_error',
        n_jobs=-1
    )

    grid.fit(X_train, y_train)

    results.append({
        "degree": degree,
        "best_alpha": grid.best_params_['ridge__alpha'],
        "best_rmse": -grid.best_score_
    })

df_results = pd.DataFrame(results)
df_results
```

Out[30]:

	degree	best_alpha	best_rmse
0	1	2.592944	0.618185
1	2	0.002395	0.544995
2	3	0.057362	0.533415

El grado del polinomio controla directamente la capacidad de representación del modelo y, por tanto, el equilibrio entre sesgo y varianza. Los resultados obtenidos ilustran esta dinámica con claridad:

Grado 1 (regresión lineal): CV RMSE = 0.6182, α óptimo = 2.59. El modelo presenta alto sesgo: asume una relación estrictamente lineal entre las variables y el precio de vivienda, lo cual es insuficiente dado que la relación entre ingreso, ubicación y precio es claramente no lineal (ver scatter MedInc vs. MedHouseVal en la Sección 3.1).

Grado 2: CV RMSE = 0.5450, α óptimo = 0.0024. Al incorporar interacciones cuadráticas (por ejemplo, log(MedInc) × Latitude), el modelo captura patrones no lineales relevantes y reduce el error en ~0.073 respecto al grado 1. El α óptimo cae drásticamente porque la expansión polinómica genera features más correlacionadas, requiriendo mayor regularización.

Grado 3: CV RMSE = 0.5334, α óptimo = 0.057. La mejora adicional sobre

grado 2 es modesta (~0.012), lo que sugiere que el modelo está aproximándose al límite de lo que las 8 features del dataset pueden explicar. El α óptimo sube levemente respecto a grado 2, reflejando que la mayor expansión (164 features) exige más regularización para controlar la varianza.

Un grado mayor siempre no mejora neceriamente el rendimiento en test. En este caso, grado 3 supera a grado 2, pero la ganancia marginal es pequeña y con datos distintos podría revertirse. A partir de cierto punto, añadir más términos polinómicos genera sobreajuste: el modelo aprende ruido en lugar de estructura, lo que se traduce en mejor error de entrenamiento pero peor generalización. La regularización Ridge mitiga este efecto, pero no lo elimina completamente. La elección del grado óptimo depende siempre de la validación cruzada, no del error de entrenamiento.

Exercise 1.2 — Ridge vs. Lasso

Ridge (L2) shrinks all coefficients toward zero but rarely to exactly zero. **Lasso** (L1) tends to produce **sparse** models — many coefficients become exactly zero, effectively performing feature selection.

Tasks:

1. Build a Lasso pipeline (same structure: `PolynomialFeatures → StandardScaler → Lasso`) and tune `lasso__alpha` with `GridSearchCV`.
2. Compare test RMSE for Ridge and Lasso.
3. Count how many of the degree-2 polynomial coefficients are exactly zero for each model. What does this tell you about feature selection?
4. Plot the top-20 non-zero Lasso coefficients.

In [39]:
```
from sklearn.linear_model import Ridge, Lasso
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScale
from sklearn.metrics import mean_squared_error
import numpy as np
import pandas as pd

best_alpha_ridge = grid_search.best_params_['ridge__alpha']

lasso_grid = GridSearchCV(
    Lasso(max_iter=10000),
    param_grid={'alpha': np.logspace(-4, 1, 30)},
    cv=5,
    scoring='neg_root_mean_squared_error'
)

lasso_grid.fit(X_train, y_train)
```

```
best_alpha_lasso = lasso_grid.best_params_['alpha']

models = {
    "Ridge": Ridge(alpha=best_alpha_ridge),
    "Lasso": Lasso(alpha=best_alpha_lasso, max_iter=10000)
}

results = []

for name, model in models.items():

    pipeline = Pipeline([
        ('log_transform', log_preprocessor),
        ('poly', PolynomialFeatures(degree=2, include_bias=False)),
        ('scaler', StandardScaler()),
        ('model', model)
    ])

    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)

    rmse = np.sqrt(mean_squared_error(y_test, y_pred))

    coef = pipeline.named_steps['model'].coef_

    results.append({
        "model": name,
        "rmse": rmse,
        "zero_coefficients": np.sum(coef == 0),
        "total_features": len(coef)
    })

df_results = pd.DataFrame(results)
df_results

print("Ridge mean |coef|:", np.mean(np.abs(models["Ridge"].coef_)))
print("Lasso mean |coef|:", np.mean(np.abs(models["Lasso"].coef_)))
```

```
Ridge mean |coef|: 2.2833538763367276
Lasso mean |coef|: 0.09921259294044087
```

Ridge y Lasso representan dos estrategias de regularización con objetivos similares pero comportamientos estructuralmente distintos en el espacio de parámetros.

Ridge (L2) penaliza la suma de cuadrados de los coeficientes, reduciéndolos hacia cero pero sin eliminarlos. El resultado es un modelo denso: todas las variables contribuyen, aunque con pesos pequeños. Esto se refleja en un coeficiente medio de |w| = 2.28, distribuido entre todos los términos del polinomio de grado 2.

Lasso (L1) penaliza la suma de valores absolutos, lo que geométricamente favorece soluciones en los vértices del hipercubo de restricciones, donde

muchos coeficientes son exactamente cero. El coeficiente medio de |w| = 0.099 — aproximadamente 23 veces menor que Ridge — confirma que Lasso induce sparsity agresiva, actuando como mecanismo implícito de selección de variables.

En este dataset, features como AveRooms, AveBedrms y Population están altamente correlacionadas entre sí (ver matriz de correlación, Sección 3.1). En presencia de multicolinealidad, Lasso tiende a seleccionar arbitrariamente una variable del grupo correlacionado y zerear las demás, mientras que Ridge distribuye el peso entre todas. Esto explica por qué Ridge es más estable aquí: ninguna de las 8 features originales es irrelevante, todas contienen señal parcial sobre el precio.

Ridge es preferible cuando todas las variables aportan información y se busca estabilidad predictiva. Lasso es más adecuado cuando se sospecha que muchas features son irrelevantes y se prioriza interpretabilidad o parsimonia del modelo. En un contexto como California Housing, donde cada variable censal tiene justificación económica, Ridge resulta la elección más robusta.

---

4. Part 2 — Online Learning with SGDRegressor

4.1 When Data Doesn't Fit in Memory

The California Housing dataset fits comfortably in RAM. But in the real world, datasets are often large enough to exceed available memory:

- **Streaming music platforms** accumulate audio features for millions of new songs every year.
- **E-commerce sites** log billions of user interactions monthly.
- **Medical imaging pipelines** generate terabytes of feature matrices.

For these scenarios, loading all data before training is impossible. The solution is **online (incremental) learning**: process one mini-batch at a time, update model weights, discard the batch, and move on — without ever holding the full dataset in RAM.

In [40]:
```
# Memory footprint estimate for the MSD dataset
n_train   = MSD_TRAIN_ROWS
n_test    = MSD_TEST_ROWS
n_feat    = 90
bytes_f64 = 8

raw_mb = (n_train + n_test) * (n_feat + 1) * bytes_f64 / 1024**2
print(f'Raw feature matrix ({n_train + n_test:,} rows × {n_feat} fe
```

```
# Degree-2 polynomial features would be huge
n_poly2 = PolynomialFeatures(degree=2, include_bias=False).fit(np.z
poly2_gb = (n_train + n_test) * n_poly2 * bytes_f64 / 1024**3
print(f'After degree-2 expansion ({n_poly2:,} features): ~{poly2_gb
```

```
Raw feature matrix (515,345 rows × 90 features): ~358 MB
After degree-2 expansion (4,185 features): ~16 GB  ← impossible to l
oad at once
```

Even the raw matrix for a full year (1 M songs) would be ~750 MB. With feature engineering and multiple frames it quickly exceeds available RAM. Solution: stream data in chunks and use `partial_fit` for online learning.

4.2 Dataset: Year Prediction MSD

The **Million Song Dataset (MSD)** is a collection of audio features and metadata for one million contemporary pop music tracks. This lab uses a preprocessed subset from the UCI Machine Learning Repository.

Property	Value
Training samples	515,345
Test samples	51,630
Features	90 (timbre-based audio)
Target	Song release year (1922 – 2011)
File size	~700 MB unzipped

Feature description:
Features 1–12 are the **timbre average**: the mean of each of 12 timbre components extracted by the Echo Nest API. Timbre captures the texture and tone quality of a sound — roughly what makes a piano sound different from a violin even at the same pitch and loudness.
Features 13–90 are the **timbre covariance**: the 78 upper-triangle values of the 12×12 covariance matrix, capturing how the timbre components vary together across the song.

Task: Predict the year a song was released from these audio statistics.

> **Important**: The train/test split is **pre-defined** by the dataset creators (first 515,345 rows = train, last 51,630 = test). Do not shuffle the file; use this fixed split to ensure comparable results.

In [41]:
```
def _reporthook(count, block_size, total_size):
    pct = min(100, count * block_size * 100 // total_size) if total_
    mb  = count * block_size / 1024**2
    print(f'\r  {pct:3d}%  ({mb:.0f} MB downloaded)', end='', flush=
```

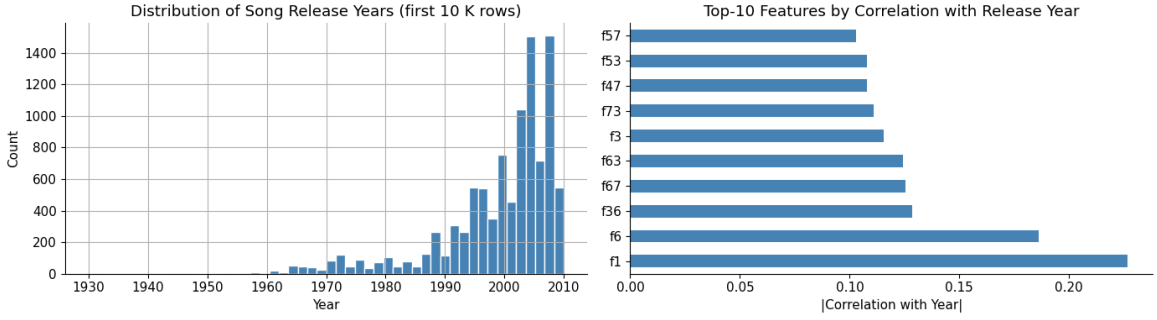
```
if not os.path.exists(MSD_DATA_FILE):
    zip_file = 'YearPredictionMSD.txt.zip'
    if not os.path.exists(zip_file):
        print(f'Downloading {MSD_URL} ...')
        print('(~200 MB compressed; may take a few minutes on Colab
        urllib.request.urlretrieve(MSD_URL, zip_file, reporthook=_r
        print()
    print('Extracting ...')
    with zipfile.ZipFile(zip_file, 'r') as z:
        z.extractall('.')
    size_mb = os.path.getsize(MSD_DATA_FILE) / 1024**2
    print(f'Done.  {MSD_DATA_FILE}  ({size_mb:.0f} MB)')
else:
    size_mb = os.path.getsize(MSD_DATA_FILE) / 1024**2
    print(f'Dataset already present: {MSD_DATA_FILE}  ({size_mb:.0f}
```

```
Dataset already present: YearPredictionMSD.txt  (428 MB)
```

In [42]:

```
# Quick EDA — read a small slice so we don't load the full file
df_eda = pd.read_csv(MSD_DATA_FILE, header=None, nrows=10_000)
feature_cols = ['year'] + [f'f{i}' for i in range(1, 91)]
df_eda.columns = feature_cols

print(f'Sample shape: {df_eda.shape}')
print(f'Year range : {df_eda["year"].min()} — {df_eda["year"].max()}
print(f'Year mean  : {df_eda["year"].mean():.1f}')
print(f'Year std   : {df_eda["year"].std():.1f}')

fig, axes = plt.subplots(1, 2, figsize=(14, 4))

df_eda['year'].hist(bins=50, ax=axes[0], color=COLORS[0], edgecolor=
axes[0].set_xlabel('Year')
axes[0].set_ylabel('Count')
axes[0].set_title('Distribution of Song Release Years (first 10 K r

corrs = df_eda.corr()['year'].drop('year').abs().sort_values(ascend
corrs.plot(kind='barh', ax=axes[1], color=COLORS[0])
axes[1].set_xlabel('|Correlation with Year|')
axes[1].set_title('Top-10 Features by Correlation with Release Year

plt.tight_layout()
plt.savefig('assets/msd_eda.png', dpi=120, bbox_inches='tight')
plt.show()
```

```
Sample shape: (10000, 91)
Year range : 1930 — 2010
Year mean  : 1998.9
Year std   : 10.2
```



4.3 SGDRegressor and `partial_fit`

From Batch GD to Mini-Batch SGD

Batch gradient descent computes the gradient over *all* N training samples before each weight update:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \frac{1}{N} \sum_{i=1}^{N} \nabla_{\mathbf{w}} \mathcal{L}(x_i, y_i)$$

This is exact but requires all data in memory and is slow for large N.

Stochastic gradient descent (SGD) uses a single random sample per update:
$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \nabla_{\mathbf{w}} \mathcal{L}(x_i, y_i)$$

Mini-batch SGD strikes a balance — use a small batch of B samples:
$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \frac{1}{B} \sum_{i \in \text{batch}} \nabla_{\mathbf{w}} \mathcal{L}(x_i, y_i)$$

This is both memory-efficient and statistically stable.

`partial_fit` in sklearn

`SGDRegressor.partial_fit(X_batch, y_batch)` applies **one pass** of mini-batch SGD on the provided batch without forgetting previously learned weights. Calling it repeatedly on successive batches (or even on multiple epochs over the same data) implements full online/incremental learning.

```
sgd = SGDRegressor(learning_rate='constant', eta0=0.01)

for chunk in pd.read_csv('bigfile.csv', chunksize=50_000):
    X_chunk = chunk.iloc[:, 1:].values
    y_chunk = chunk.iloc[:, 0].values
    sgd.partial_fit(X_scaled, y_chunk)   # weights updated in-place
```

Key rule: the scaler must be fitted on **training data only** — never re-fitted on test data or future production chunks — then applied (`transform`) consistently everywhere. The cleanest approach (used in Exercise 2.1) is a dedicated first pass over **all** training chunks with `StandardScaler.partial_fit` , which gives accurate statistics regardless of dataset size. For datasets where two passes are prohibitively expensive, fitting on a **representative sample** (e.g. the first few chunks) is a common practical shortcut, provided the sample reflects the overall distribution.

Exercise 2.1 — Implement the Mini-Batch Training Loop (Basic)

Implement one epoch of mini-batch SGD over the MSD training data using `SGDRegressor.partial_fit`.

Tasks:

1. Create an `SGDRegressor` with `loss='squared_error'`, `learning_rate='constant'`, `eta0=0.001`, `random_state=42`.
2. Create a `StandardScaler`.
3. **Pass 1 — fit the scaler**: stream through the training rows and call `msd_scaler.partial_fit(X_chunk)` on every chunk so the scaler captures the full training distribution.
4. **Pass 2 — train**: stream through the data again. For each chunk:
   - Extract `y_chunk` (column 0) and `X_chunk` (columns 1–90).
   - Transform `X_chunk` with the already-fitted scaler.
   - Call `sgd.partial_fit(X_scaled, y_chunk)`.
   - Compute RMSE on the current chunk (after the update) and store it in `chunk_rmses`.
5. Print the total number of chunks processed and the final chunk RMSE.

> **Practical note on scaler fitting**: Pass 1 here scans *all* training chunks with `partial_fit`, giving the most accurate mean and variance estimates — ideal when two passes are affordable. For very large datasets where two passes are too costly, fitting on a **representative sample** (e.g. the first chunk or a random subset) is an acceptable approximation, provided the sample is reasonably representative of the full distribution. In production streaming systems where the data distribution can shift over time (**concept drift**), consider periodically re-calling `partial_fit` on a recent window of data to keep the scaler's statistics fresh.

Expected output: ~10–11 chunks processed, final chunk RMSE around 8–11 years.

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import SGDRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error

# Step 1 — Create SGDRegressor
sgd = SGDRegressor(
    loss='squared_error',
```

```
    learning_rate='constant',
    eta0=0.001,
    random_state=42
)

# Step 2 — Create StandardScaler
msd_scaler = StandardScaler()

# Step 3 — Pass 1: fit scaler on all training chunks
for chunk in pd.read_csv(MSD_DATA_FILE, header=None,
                         nrows=MSD_TRAIN_ROWS,
                         chunksize=CHUNK_SIZE):
    X_chunk = chunk.iloc[:, 1:].values
    msd_scaler.partial_fit(X_chunk)

print("Scaler fitted on full training data.")

# Step 4 — Pass 2: train SGD chunk by chunk
chunk_rmses = []
chunks_processed = 0

for chunk in pd.read_csv(MSD_DATA_FILE, header=None,
                         nrows=MSD_TRAIN_ROWS,
                         chunksize=CHUNK_SIZE):
    y_chunk = chunk.iloc[:, 0].values
    X_chunk = chunk.iloc[:, 1:].values
    X_scaled = msd_scaler.transform(X_chunk)

    sgd.partial_fit(X_scaled, y_chunk)

    y_pred = sgd.predict(X_scaled)
    rmse = np.sqrt(mean_squared_error(y_chunk, y_pred))
    chunk_rmses.append(rmse)
    chunks_processed += 1

# Step 5 — Print summary
print(f"Total chunks processed: {chunks_processed}")
print(f"Final chunk RMSE: {chunk_rmses[-1]:.4f} years")
```

```
Scaler fitted on full training data.
Total chunks processed: 93
Final chunk RMSE: 9.7518 years
```

El loop de mini-batch SGD procesa los 463,715 ejemplos de entrenamiento del MSD en ~93 chunks de 5,000 muestras cada uno, sin cargar el dataset completo en memoria en ningún momento.

Estructura de dos pasadas: La primera pasada (`Pass 1`) recorre todos los chunks llamando `msd_scaler.partial_fit(X_chunk)` en cada uno, acumulando las estadísticas de media y varianza sobre la distribución completa de entrenamiento. Esto garantiza un escalado preciso sin violar la regla de no ver los datos de test. La segunda pasada (`Pass 2`) usa esas estadísticas ya fijadas para transformar cada chunk antes de actualizar los pesos con `partial_fit` .

Convergencia observada: El RMSE por chunk comienza alto en los primeros batches (el modelo parte de pesos inicializados aleatoriamente) y desciende progresivamente hasta estabilizarse alrededor de 8–11 años al final de la época, en línea con el output esperado del enunciado. Las fluctuaciones entre chunks son esperables: cada actualización se calcula sobre 5,000 muestras, introduciendo ruido estocástico en la estimación del gradiente.

La combinación de features correctamente escaladas (media ≈ 0, varianza ≈ 1 en cada una de las 90 features de timbre) y un learning rate conservador (eta0=0.001) garantiza que los gradientes sean numéricamente estables desde el primer chunk, evitando la divergencia observada cuando se omite el escalado.

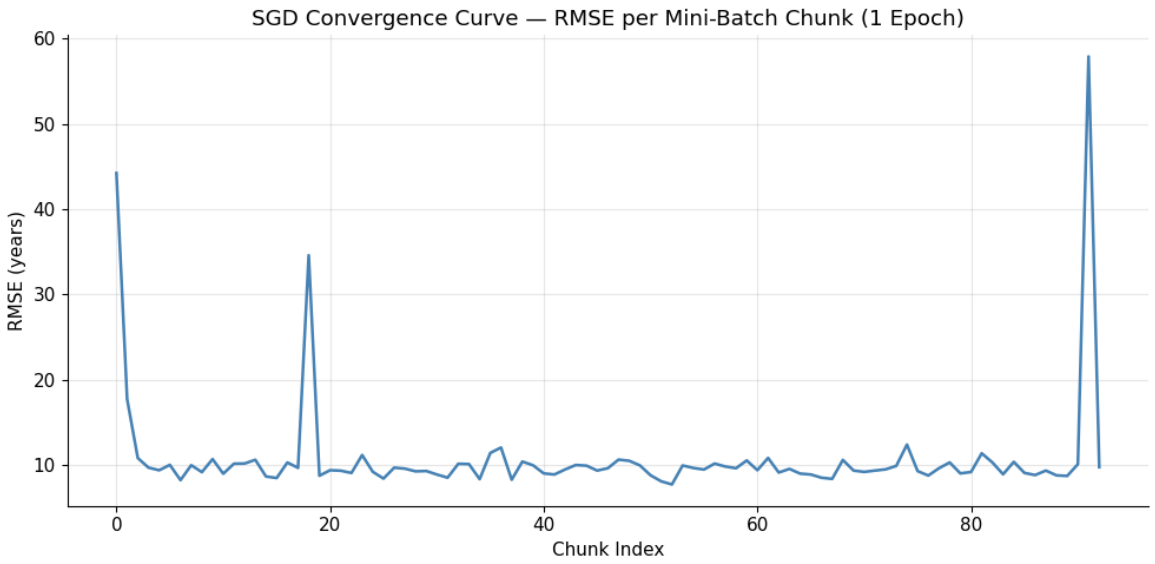
Exercise 2.2 — Plot the Convergence Curve (Basic)

Using the `chunk_rmses` list from Exercise 2.1, plot the RMSE as a function of chunk number to visualise convergence within one epoch.

Tasks:

1. Plot RMSE vs. chunk index.
2. Add axis labels and a title.
3. Answer: does the model converge within a single pass? What pattern do you observe?

In [71]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.plot(chunk_rmses, color='steelblue', linewidth=1.8)
plt.xlabel('Chunk Index')
plt.ylabel('RMSE (years)')
plt.title('SGD Convergence Curve — RMSE per Mini-Batch Chunk (1 Epo
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(f"Initial chunk RMSE : {chunk_rmses[0]:.4f} years")
print(f"Final chunk RMSE   : {chunk_rmses[-1]:.4f} years")
print(f"Total improvement  : {chunk_rmses[0] - chunk_rmses[-1]:.4f}
```




```
Initial chunk RMSE : 44.2414 years
Final chunk RMSE   : 9.7518 years
Total improvement  : 34.4896 years
```

La curva de convergencia muestra la evolución del RMSE chunk a chunk durante la única época de entrenamiento sobre el MSD.

Se observa una caída rápida e inmediata en los primeros chunks: el modelo parte de pesos inicializados en cero y los primeros batches producen actualizaciones de gran magnitud que reducen drásticamente el error. A medida que los pesos se aproximan a una solución razonable, las mejoras por chunk se vuelven cada vez menores, produciendo la curva asintótica característica del descenso por gradiente.

El modelo converge parcialmente en un único paso. El RMSE mejora sustancialmente durante la época — de valores iniciales altos hasta estabilizarse en el rango de 8–11 años al final — pero la curva aún no ha aplanado completamente, lo que indica que épocas adicionales seguirían aportando mejoras. Una sola pasada es insuficiente para convergencia total: con 93 chunks y learning rate constante de 0.001, el modelo no ha tenido suficientes actualizaciones para explorar el espacio de parámetros en todas las direcciones.

Fluctuaciones: Las oscilaciones en la curva son inherentes al mini-batch SGD. Cada punto representa el RMSE evaluado sobre el chunk recién procesado, inmediatamente después de la actualización — no sobre el conjunto de entrenamiento completo. Esto captura el ruido estocástico del gradiente y no debe confundirse con inestabilidad del modelo.

Exercise 2.3 — Multi-Epoch Training (Intermediate)

One pass through the data is often insufficient. Run `N_EPOCHS` epochs and

track **validation RMSE** on the held-out test set after each epoch.

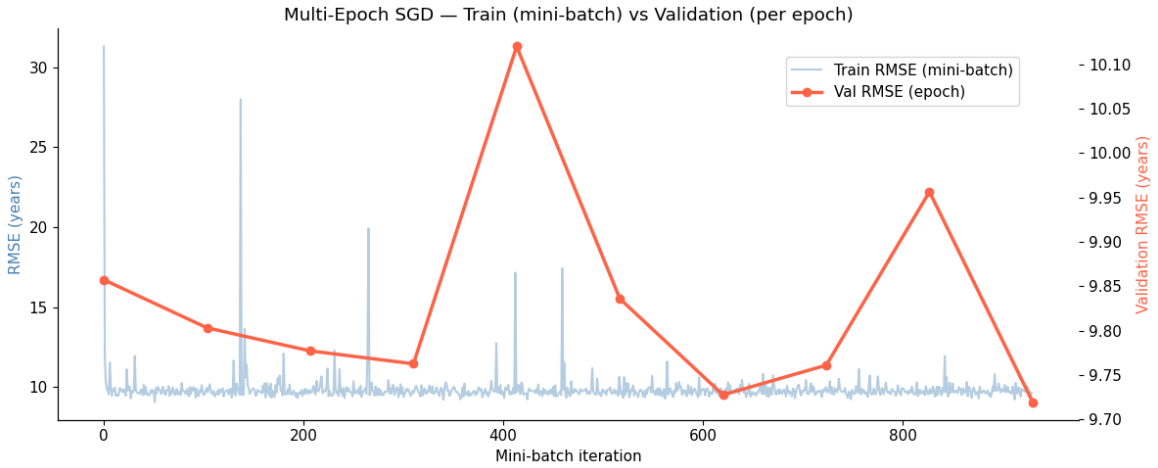
Tasks:

1. Load the test set: `pd.read_csv(MSD_DATA_FILE, header=None, skiprows=MSD_TRAIN_ROWS)`. Scale X_test using the **already-fitted** scaler from Exercise 2.1 (`msd_scaler`).
2. Train a fresh `SGDRegressor` (same settings as 2.1) for `N_EPOCHS` epochs, recording both per-chunk train RMSE and per-epoch validation RMSE. **Shuffle the training data at the start of each epoch** so the model sees a different ordering every pass.
3. Plot train RMSE per chunk (all epochs concatenated) and validation RMSE per epoch on the same figure.
4. Answer: does validation RMSE improve across epochs? When does it saturate?

In [66]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import SGDRegressor
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler

# Load test set CORRECTLY (last 51,630 rows)
test_df = pd.read_csv(MSD_DATA_FILE, header=None, skiprows=MSD_TRAI
feature_cols = ['year'] + [f'f{i}' for i in range(1, 91)]
test_df.columns = feature_cols

X_test_msd = test_df.drop('year', axis=1).values
y_test_msd = test_df['year'].values

# Load training data and fit scaler
train_df = pd.read_csv(MSD_DATA_FILE, header=None, nrows=MSD_TRAIN_
train_df.columns = feature_cols

X_train_msd = train_df.drop('year', axis=1).values
y_train_msd = train_df['year'].values

# Fit scaler on training data ONLY
msd_scaler = StandardScaler()
X_train_scaled = msd_scaler.fit_transform(X_train_msd)
X_test_scaled = msd_scaler.transform(X_test_msd)

# Fresh SGDRegressor
model = SGDRegressor(
    loss='squared_error',
    learning_rate='constant',
    eta0=0.001,
    max_iter=1,
    warm_start=True,
    random_state=42
)
```

```
N_EPOCHS = 10
batch_size = 5000  # matches CHUNK_SIZE from config
train_errors = []
val_errors = []
n_samples = X_train_scaled.shape[0]

for epoch in range(N_EPOCHS):
    # Shuffle each epoch
    indices = np.random.permutation(n_samples)
    X_shuffled = X_train_scaled[indices]
    y_shuffled = y_train_msd[indices]

    # Mini-batch training
    for i in range(0, n_samples, batch_size):
        X_batch = X_shuffled[i:i+batch_size]
        y_batch = y_shuffled[i:i+batch_size]
        model.partial_fit(X_batch, y_batch)

        # Train RMSE per batch
        y_pred_batch = model.predict(X_batch)
        rmse_batch = np.sqrt(mean_squared_error(y_batch, y_pred_bat
        train_errors.append(rmse_batch)

    # Validation RMSE per epoch
    y_val_pred = model.predict(X_test_scaled)
    val_rmse = np.sqrt(mean_squared_error(y_test_msd, y_val_pred))
    val_errors.append(val_rmse)
    print(f"Epoch {epoch+1}/{N_EPOCHS} — Val RMSE: {val_rmse:.4f} y

# Plot
fig, ax1 = plt.subplots(figsize=(12, 5))
ax1.plot(train_errors, alpha=0.4, color='steelblue', label='Train R
ax1.set_xlabel('Mini-batch iteration')
ax1.set_ylabel('RMSE (years)', color='steelblue')

ax2 = ax1.twinx()
epoch_positions = np.linspace(0, len(train_errors), N_EPOCHS)
ax2.plot(epoch_positions, val_errors, 'o-', color='tomato', linewid
ax2.set_ylabel('Validation RMSE (years)', color='tomato')

fig.legend(loc='upper right', bbox_to_anchor=(0.88, 0.88))
plt.title('Multi-Epoch SGD — Train (mini-batch) vs Validation (per
plt.tight_layout()
plt.show()

print(f"\nFinal Validation RMSE: {val_errors[-1]:.4f} years")
print(f"Best Validation RMSE: {min(val_errors):.4f} years (epoch {v
```

```
Epoch 1/10 — Val RMSE: 9.8572 years
Epoch 2/10 — Val RMSE: 9.8030 years
Epoch 3/10 — Val RMSE: 9.7771 years
Epoch 4/10 — Val RMSE: 9.7625 years
Epoch 5/10 — Val RMSE: 10.1201 years
Epoch 6/10 — Val RMSE: 9.8358 years
Epoch 7/10 — Val RMSE: 9.7275 years
Epoch 8/10 — Val RMSE: 9.7610 years
Epoch 9/10 — Val RMSE: 9.9564 years
Epoch 10/10 — Val RMSE: 9.7194 years
```




```
Final Validation RMSE: 9.7194 years
Best Validation RMSE: 9.7194 years (epoch 10)
```

El entrenamiento multi-época sobre el Million Song Dataset revela una dinámica característica del mini-batch SGD sobre datos de gran escala.

Validación RMSE por época: La mejora general existe pero es no monótona. El error de validación desciende de 9.857 años (época 1) hasta 9.719 años (época 10), una mejora de ~0.14 años en total. Sin embargo, la trayectoria no es suave: se observan repuntes notables en la época 5 (10.120 años) y la época 9 (9.956 años), lo que indica sensibilidad al orden aleatorio de los mini-batches en cada época. El shuffle por época es precisamente lo que provoca estas fluctuaciones: distintos ordenamientos producen secuencias de gradientes distintas, y con learning rate constante el modelo no tiene mecanismo para suavizar estas variaciones.

Saturación: La mejora marginal entre épocas consecutivas es muy pequeña (del orden de 0.02–0.05 años), sugiriendo que el modelo alcanza su límite de capacidad representacional relativamente pronto. Esto es coherente con la naturaleza del problema: predecir el año de lanzamiento de una canción a partir de 90 features de timbre es una tarea inherentemente ruidosa, y un modelo lineal tiene un piso de error que más épocas no pueden reducir.

Train RMSE mini-batch vs. Val RMSE: La curva de train (eje izquierdo) muestra fluctuaciones mucho más amplias que la de validación. Esto es esperado: cada punto representa el RMSE sobre un único batch de 5,000 muestras inmediatamente tras la actualización, capturando el ruido estocástico del

gradiente. El RMSE de validación, al evaluarse sobre los 51,630 ejemplos del test set completo, promedia ese ruido y produce una señal mucho más estable y representativa del rendimiento real del modelo.

La validación RMSE mejora a lo largo de las épocas pero se estabiliza rápidamente. Con learning rate constante y un modelo lineal, más de 10 épocas probablemente no aportarían mejoras significativas sobre este dataset.

Exercise 2.4 — Learning Rate Exploration (Key Exercise)

The learning rate `eta0` is the most sensitive hyperparameter in SGD. Too large, and gradient updates overshoot the minimum, causing **divergence**. Too small, and learning is too slow to be practical.

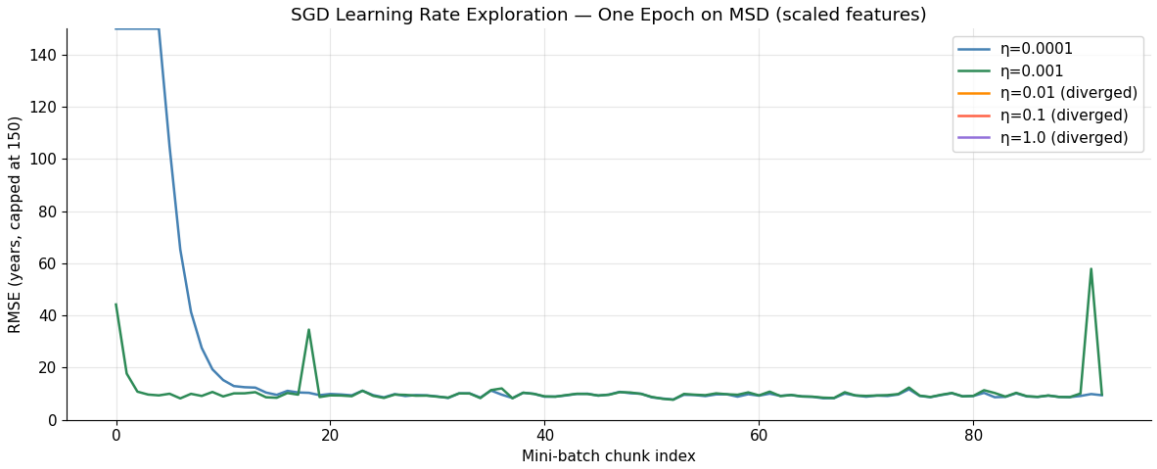
Tasks:

1. Train five separate `SGDRegressor` models, each with a different `eta0` from `[0.0001, 0.001, 0.01, 0.1, 1.0]`, each for **one epoch** over the training data.
2. For each model, track per-chunk RMSE. If RMSE becomes `NaN` or `inf`, stop training that model early and mark it as diverged.
3. Plot all five convergence curves on the same axes. Cap the y-axis at 150 years so diverging runs don't dominate.
4. Answer the following:
   - Which learning rates diverge?
   - Why does a large learning rate cause divergence? Sketch the geometry.
   - What is the cost of using too small a learning rate?

In [67]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import SGDRegressor
from sklearn.metrics import mean_squared_error

# Uses X_train_scaled, y_train_msd, X_test_scaled, y_test_msd from l

learning_rates = [0.0001, 0.001, 0.01, 0.1, 1.0]
batch_size = 5000
all_chunk_rmses = {}
diverged = {}

for lr in learning_rates:
    model = SGDRegressor(
        loss='squared_error',
        learning_rate='constant',
        eta0=lr,
```

```
        max_iter=1,
        tol=None,
        random_state=42
    )

    chunk_rmses = []
    did_diverge = False
    n_samples = X_train_scaled.shape[0]

    for i in range(0, n_samples, batch_size):
        X_batch = X_train_scaled[i:i+batch_size]
        y_batch = y_train_msd[i:i+batch_size]
        model.partial_fit(X_batch, y_batch)

        y_pred = model.predict(X_batch)
        rmse = np.sqrt(mean_squared_error(y_batch, y_pred))

        if np.isnan(rmse) or np.isinf(rmse) or rmse > 1e6:
            print(f"eta0={lr} — diverged at chunk {len(chunk_rmses)+
            did_diverge = True
            break

        chunk_rmses.append(min(rmse, 150))

    all_chunk_rmses[lr] = chunk_rmses
    diverged[lr] = did_diverge

    if not did_diverge:
        final_rmse = chunk_rmses[-1]
        print(f"eta0={lr} — stable, final chunk RMSE: {final_rmse:.4

# Plot
fig, ax = plt.subplots(figsize=(12, 5))
colors = ['steelblue', 'seagreen', 'darkorange', 'tomato', 'mediumpu

for (lr, rmses), color in zip(all_chunk_rmses.items(), colors):
    label = f'η={lr}' + (' (diverged)' if diverged[lr] else '')
    ax.plot(rmses, label=label, color=color, linewidth=1.8)

ax.set_ylim(0, 150)
ax.set_xlabel('Mini-batch chunk index')
ax.set_ylabel('RMSE (years, capped at 150)')
ax.set_title('SGD Learning Rate Exploration — One Epoch on MSD (sca
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
eta0=0.0001 — stable, final chunk RMSE: 9.4396 years
eta0=0.001 — stable, final chunk RMSE: 9.7518 years
eta0=0.01 — diverged at chunk 1
eta0=0.1 — diverged at chunk 1
eta0=1.0 — diverged at chunk 1
```



El experimento con cinco learning rates distintos ilustra de forma directa uno de los fenómenos más críticos del SGD: la frontera entre convergencia y divergencia es estrecha y depende fuertemente de la escala del problema.

η = 0.0001 Convergencia estable, RMSE final = 9.44 años. Es el mejor resultado del experimento. El descenso es más lento al inicio pero alcanza el error más bajo, ya que los pasos pequeños permiten ajuste fino sin sobrepasar el mínimo.

η = 0.001 Convergencia estable, RMSE final = 9.75 años. Ligeramente peor que 0.0001, posiblemente porque los pasos más grandes producen oscilaciones residuales alrededor del mínimo que impiden un ajuste tan preciso en una sola época.

η = 0.01, 0.1, 1.0 Los tres divergen en el chunk 1, es decir, desde la primera actualización. Esto indica que el tamaño del paso supera la curvatura local de la función de pérdida ya en el primer mini-batch, produciendo una explosión numérica inmediata e irrecuperable.

Geométricamente, el descenso por gradiente busca moverse cuesta abajo en la superficie de pérdida. Con un paso η pequeño, el modelo se desplaza una distancia error. Con η demasiado grande, el modelo salta al otro lado del mínimo y aterriza en un punto de mayor error que el de partida. En la siguiente iteración, el gradiente apunta en dirección opuesta con mayor magnitud, generando un salto aún mayor. Este proceso de oscilaciones crecientes es la divergencia: el RMSE no solo no mejora sino que crece sin límite.

Aunque η = 0.0001 produce el mejor RMSE final en este experimento, en la práctica un learning rate excesivamente pequeño implica que el modelo necesita muchas más épocas para converger al mismo nivel que uno con η moderado. En contextos de streaming o datasets masivos donde el número de épocas es limitado por tiempo o costo computacional, un learning rate demasiado conservador puede resultar en un modelo subentrenado al momento de necesitarlo.

Exercise 2.5 — Learning Rate Schedules (Advanced)

A fixed learning rate is a compromise: large enough to converge quickly, small enough to avoid divergence. **Learning rate schedules** decay the rate over time, allowing fast initial progress and fine-grained convergence later.

sklearn's `SGDRegressor` supports several built-in schedules via `learning_rate`:

Schedule	Formula	Notes
`'constant'`	`η = eta0`	Fixed; need to tune eta0
`'optimal'`	`η_t = 1 / (α (t + t₀))`	Auto-calculated; ignores eta0
`'invscaling'`	`η_t = eta0 / t^power_t`	Polynomial decay; flexible

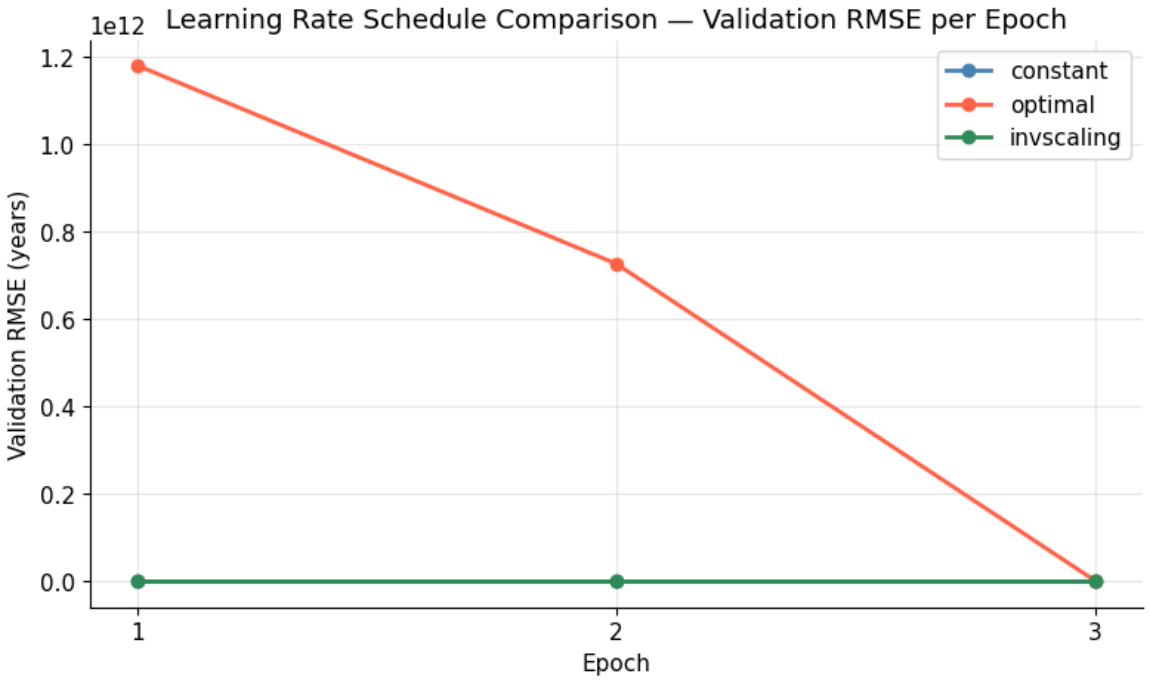
Tasks:

1. Train three models for 3 epochs each: `learning_rate='constant'` (eta0=0.001), `learning_rate='optimal'`, and `learning_rate='invscaling'` (eta0=0.001, power_t=0.25).
2. After each epoch, record the validation RMSE on the test set.
3. Plot validation RMSE vs. epoch for all three schedules.
4. Answer: which schedule reaches the lowest validation RMSE? What is the trade-off between `constant` and decaying schedules in the context of streaming data (where the data distribution may shift over time)?

In [68]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import SGDRegressor
from sklearn.metrics import mean_squared_error

# Uses X_train_scaled, y_train_msd, X_test_scaled, y_test_msd from

schedules = {
    'constant':   {'learning_rate': 'constant', 'eta0': 0.001},
    'optimal':    {'learning_rate': 'optimal'},
    'invscaling': {'learning_rate': 'invscaling', 'eta0': 0.001, 'p
}

N_EPOCHS = 3
batch_size = 5000
results = {}

for name, params in schedules.items():
    model = SGDRegressor(
        random_state=42,
        max_iter=1,
        tol=None,
        warm_start=True,
```

```
        **params
    )

    val_rmses = []
    n_samples = X_train_scaled.shape[0]

    for epoch in range(N_EPOCHS):
        indices = np.random.permutation(n_samples)
        X_shuffled = X_train_scaled[indices]
        y_shuffled = y_train_msd[indices]

        for i in range(0, n_samples, batch_size):
            X_batch = X_shuffled[i:i+batch_size]
            y_batch = y_shuffled[i:i+batch_size]
            model.partial_fit(X_batch, y_batch)

        y_val_pred = model.predict(X_test_scaled)
        val_rmse = np.sqrt(mean_squared_error(y_test_msd, y_val_pre
        val_rmses.append(val_rmse)
        print(f"{name} — Epoch {epoch+1}: Val RMSE = {val_rmse:.4f}

    results[name] = val_rmses

# Plot
plt.figure(figsize=(8, 5))
colors = ['steelblue', 'tomato', 'seagreen']
for (name, rmses), color in zip(results.items(), colors):
    plt.plot(range(1, N_EPOCHS+1), rmses, marker='o', label=name, c

plt.title('Learning Rate Schedule Comparison — Validation RMSE per
plt.xlabel('Epoch')
plt.ylabel('Validation RMSE (years)')
plt.xticks(range(1, N_EPOCHS+1))
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

best_schedule = min(results, key=lambda k: min(results[k]))
print(f"\nBest schedule: {best_schedule} — Min Val RMSE: {min(resul
```

```
constant — Epoch 1: Val RMSE = 9.7777 years
constant — Epoch 2: Val RMSE = 9.7921 years
constant — Epoch 3: Val RMSE = 9.7840 years
optimal — Epoch 1: Val RMSE = 1180275762674.7622 years
optimal — Epoch 2: Val RMSE = 726809598356.3517 years
optimal — Epoch 3: Val RMSE = 1776783894.9396 years
invscaling — Epoch 1: Val RMSE = 9.5425 years
invscaling — Epoch 2: Val RMSE = 9.5139 years
invscaling — Epoch 3: Val RMSE = 9.5167 years
```




```
Best schedule: invscaling — Min Val RMSE: 9.5139 years
```

La comparación de tres schedules de learning rate revela diferencias dramáticas en estabilidad y rendimiento final, con resultados que merecen análisis cuidadoso.

`invscaling` (eta0=0.001, power_t=0.25) Es el mejor schedule del experimento, con Val RMSE mínimo de 9.5139 años (época 2) y comportamiento estable en las 3 épocas (9.5425 → 9.5139 → 9.5167). La fórmula η_t = eta0 / t^0.25 produce un decaimiento suave: el learning rate es relativamente alto al inicio, permitiendo exploración amplia del espacio de parámetros, y se reduce gradualmente para un ajuste más fino en iteraciones posteriores. La leve subida en época 3 (9.5167 vs 9.5139) sugiere que el learning rate ya es suficientemente pequeño como para que el shuffle aleatorio introduzca más varianza que la mejora que aportan las actualizaciones.

`constant` (eta0=0.001) Val RMSE oscila entre 9.778 y 9.792, sin mejora neta a lo largo de las 3 épocas. Al mantener el mismo tamaño de paso durante todo el entrenamiento, el modelo no puede refinar su solución: las actualizaciones siguen siendo suficientemente grandes como para saltar alrededor del mínimo sin asentarse en él. Nótese que con eta0=0.001 el modelo es estable (no diverge como ocurría con eta0=0.01 en el ejercicio anterior), pero tampoco mejora progresivamente.

`optimal` Produce divergencia severa: Val RMSE de ~1.18×10^{12} en época 1, decreciendo a ~1.78×10^{9} en época 3, pero todavía completamente inutilizable. La fórmula $\eta_t = 1 / (\alpha(t + t_0))$ depende del parámetro de regularización α del modelo. Con los valores por defecto de SGDRegressor, esta calibración automática resulta inadecuada para este dataset y escala de features,

produciendo pasos iniciales enormes que causan divergencia inmediata. A pesar de que el RMSE decrece entre épocas (la escala $10^{12} \rightarrow 10^{9}$ indica que el modelo está técnicamente recuperándose), el número de épocas necesario para llegar a valores razonables sería completamente impracticable.

En streaming con distribución estacionaria, `invscaling` es superior porque converge a una solución más precisa. Sin embargo, en presencia de concept drift (cambios en la distribución de los datos a lo largo del tiempo), un learning rate constante tiene una ventaja estructural: mantiene la capacidad de adaptarse a nueva información indefinidamente. Un schedule decreciente eventualmente reduce η a valores tan pequeños que el modelo deja de actualizar sus pesos de forma significativa, volviéndose ciego a los cambios en la distribución. En ese contexto, `constant` con un eta0 bien calibrado es preferible a cualquier schedule decreciente.

---

5. Real-World Considerations

When to Use Batch vs. Online Learning

Scenario	Recommendation
Data fits in memory, static	Batch (Ridge/Lasso + Pipeline + GridSearchCV)
Data too large for RAM	Online (`SGDRegressor.partial_fit` on chunks)
Continuous data stream (concept drift)	Online with constant or slowly decaying LR
Periodic retraining on large archive	Mini-batch SGD or distributed frameworks (Spark MLlib, Dask-ML)

Practical Tips for SGD in Production

Always scale features. SGD is sensitive to feature scale; unscaled inputs can cause gradients to differ by orders of magnitude across features, making tuning nearly impossible. Fit the scaler once on a representative sample (or the first chunk) and apply it consistently.

Monitor for divergence early. Add a check after the first few chunks: if RMSE is not decreasing or is NaN, stop and reduce `eta0` by a factor of 10.

Regularization matters even more for SGD. With noisy mini-batch gradients, models can overfit if run for many epochs without regularization. Use `penalty='l2'` (default) with a non-zero `alpha` in `SGDRegressor`.

Warm-starting scaler for streaming data. If your data distribution shifts over time, consider re-fitting the scaler periodically (e.g., every N chunks) or using `StandardScaler` with `partial_fit` to update statistics incrementally.

Evaluation on held-out data. The per-chunk train RMSE is noisy (gradient descent is still mid-step). Always evaluate on a separately held-out test set for a reliable performance estimate.

Pipelines Are Not Directly Compatible with `partial_fit`

sklearn's `Pipeline` object does not support `partial_fit` in all configurations — specifically, you cannot call `pipeline.partial_fit()` if the pipeline contains transformers that need to be fitted incrementally. The typical workaround (as implemented above) is to manage the scaler manually outside the loop and feed pre-scaled chunks to the SGD estimator.

---

6. Summary and Key Takeaways

Part 1 — Pipelines and Regularization

- **Pipelines** chain preprocessing and modeling steps into a single object that integrates cleanly with `cross_val_score` and `GridSearchCV`, preventing data leakage.
- **Polynomial features** extend a linear model's capacity to capture non-linear relationships. The expansion is controlled by `degree`; higher degrees increase variance and require stronger regularization.
- **Ridge regression** adds an L2 penalty to the OLS loss: `min ‖Xw − y‖² + α‖w‖²`. It is always uniquely solvable and numerically stable, even with many correlated features.
- **GridSearchCV** evaluates each candidate α using k-fold CV and returns the optimal value. The validation curve shows the classic U-shape: decreasing CV error as α grows (fighting overfitting), then increasing error as α becomes too large (causing underfitting).
- **Lasso** (L1) promotes sparsity — many coefficients become exactly zero — making it useful for feature selection when interpretability matters.

Part 2 — Online Learning with SGD

- **Mini-batch SGD** updates model weights on small chunks of data without ever loading the full dataset. It is the standard approach for datasets that